

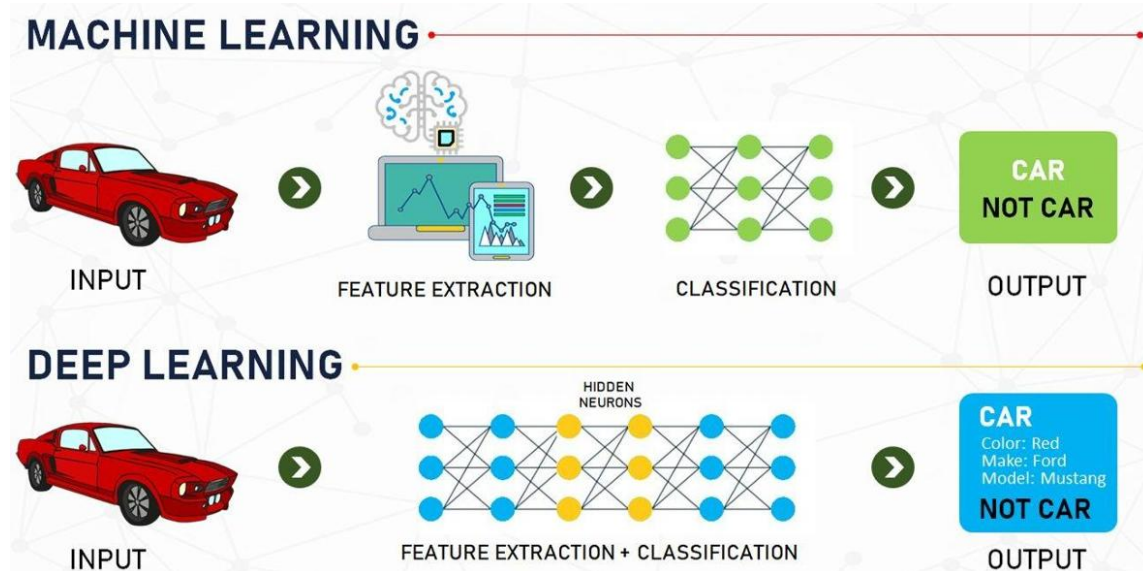
CHAPTER 4: KIẾN THỨC CHUNG VỀ NEURAL NETWORK (MẠNG NƠ-RON)

Mục lục

4.1. Giới thiệu chung về Machine Learning và Deep Learning	2
4.1.1. So sánh quy trình làm việc của Machine Learning và Deep Learning	2
4.1.2. Deep Learning for Computer Vision	3
4.1.3. Neural Network và Deep Learning	4
4.2. Linear Classifier – Bộ phân loại tuyến tính	5
4.3. Từ Linear Classifier đến Neural Network	7
4.3.1. Một số quy ước ký hiệu	8
4.3.2. Activation function – Hàm kích hoạt	9
4.3.3. Neural Network Architecture	10
4.4. Huấn luyện Mạng Neural	12
4.4.1. Quy trình huấn luyện	12
4.4.2. Loss Function (Hàm mất mát)	13
4.4.3. Gradient & Gradient Descent	16
4.4.3.1. Gradient là gì?	16
4.4.3.2. Gradient Descent	18
4.4.3.3. Momentum (Đà)	21
4.4.3.4. Adaptive Gradient Descent (Adagrad)	22
4.4.3.5. Root Mean Square Propagation (RMSProp)	23
4.4.4. BackPropagation (Lan truyền ngược)	24
4.5. Các vấn đề và giải pháp khi huấn luyện mạng sâu	29
4.5.1. Vanishing Gradient (Gradient triệt tiêu)	29
4.5.2. Exploding gradients (Gradient bùng nổ)	30
4.5.2. Cách khắc phục Vanishing/Exploding Gradient	31
4.6. Các chủ đề nâng cao	36
4.6.1. Hai trường phái chính trong Machine Learning	36
4.6.2. Generative Adversarial Networks (GANs) (Mạng Sinh Đối Nghịch)	37
4.6.3. Recurrent Neural Network (RNNs) (Mạng Neural Hồi quy)	41
4.6.3.1. Long Short-Term Memory (LSTM)	44
4.6.3.2. Gated Recurrent Unit (GRU)	46
4.7. Batch size, Iteration & Epoch	46
4.8. Fine-tune (tinh chỉnh mô hình)	47

4.1. Giới thiệu chung về Machine Learning và Deep Learning

4.1.1. So sánh quy trình làm việc của Machine Learning và Deep Learning



- **Machine Learning:**

- **Feature Extraction (Trích xuất đặc trưng):**

- Đây là bước quan trọng và **thủ công**.
 - **Con người** (kỹ sư, nhà khoa học) phải tự định nghĩa và viết code để trích xuất các đặc trưng hữu ích từ ảnh như: hình dạng bánh xe, sự hiện diện của cửa sổ, đường cong của thân xe, v.v.

- **Classification (Phân loại):** Dựa trên các đặc trưng đã trích xuất, một mô hình ML đơn giản (như SVM, Logistic Regression, hoặc một mạng Neural nhỏ) sẽ học cách phân loại đó là "CAR" (xe hơi) hay "NOT CAR" (không phải xe hơi).

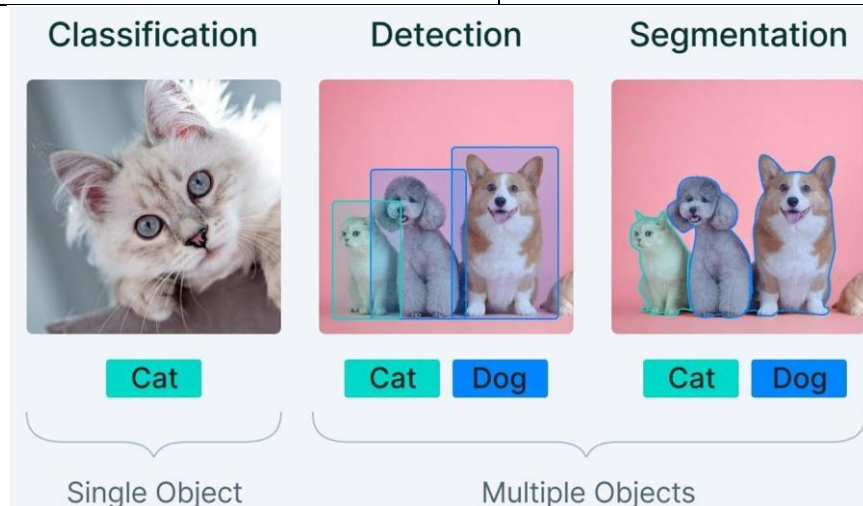
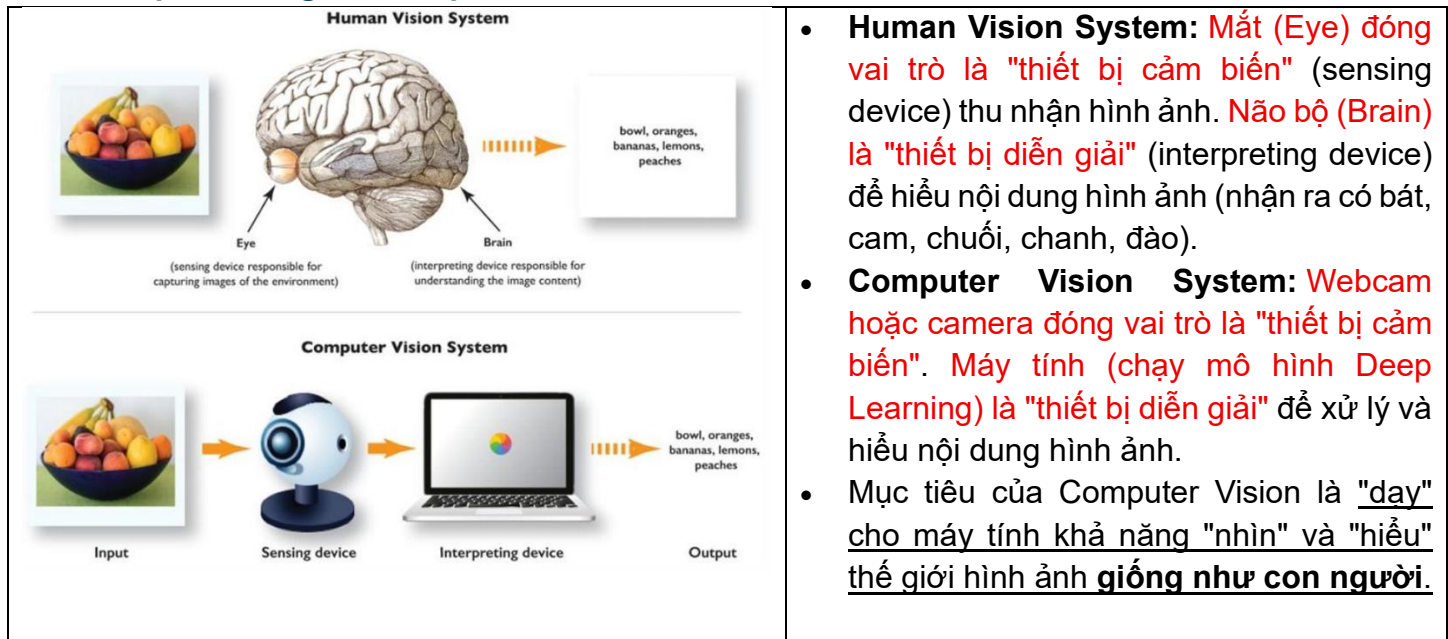
- **Deep Learning:** Quy trình được **tự động hóa** và tích hợp.

- **Feature Extraction + Classification:**

- Một mạng neural network sâu (có nhiều lớp ẩn - "hidden neurons") sẽ **tự động học** các đặc trưng cần thiết từ dữ liệu ảnh.
 - Các lớp đầu tiên có thể học các đặc trưng cơ bản như cạnh, góc, màu sắc.
 - Các lớp sâu hơn sẽ kết hợp chúng lại để học các đặc trưng phức tạp hơn như bánh xe, đèn pha, và cuối cùng là toàn bộ khái niệm về "xe hơi".
 - **Quá trình trích xuất đặc trưng và phân loại diễn ra trong cùng một mô hình.**

- Sự khác biệt cốt lõi giữa ML và DL là **tự động hóa việc trích xuất đặc trưng**. Điều này giúp DL vượt trội trong các bài toán với dữ liệu phi cấu trúc phức tạp như hình ảnh, âm thanh, văn bản, nơi việc trích xuất đặc trưng thủ công rất khó khăn và không hiệu quả. **DL cũng có thể cho ra kết quả chi tiết hơn**, như "Color: Red, Make: Ford, Model: Mustang".

4.1.2. Deep Learning for Computer Vision



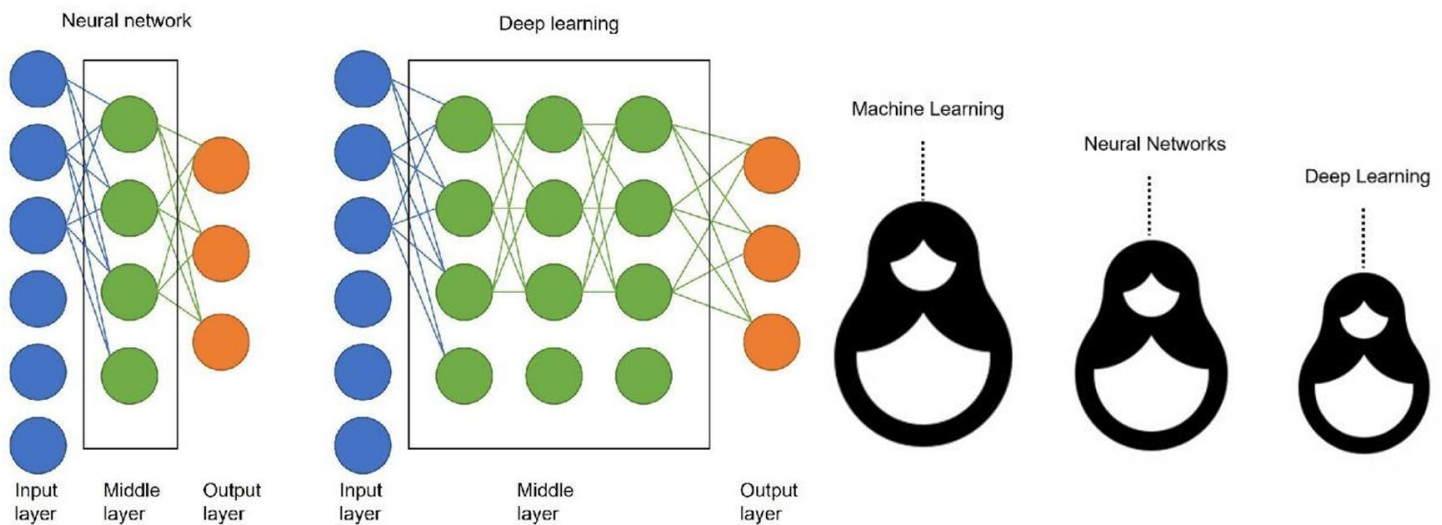
• Các tác vụ chính

- **Classification:** Trả lời câu hỏi **"Trong ảnh có gì?"**. Mô hình chỉ cần đưa ra một nhãn duy nhất cho toàn bộ bức ảnh (ví dụ: "Cat"). Áp dụng cho ảnh chỉ có một đối tượng chính.
- **Detection:** Trả lời câu hỏi **"Có những gì và chúng ở đâu?"**. Mô hình không chỉ nhận dạng các đối tượng mà còn vẽ một hộp giới hạn (bounding box) xung quanh mỗi đối tượng (ví dụ: vẽ hộp quanh con mèo và ghi nhãn "Cat", vẽ hộp quanh con chó và ghi nhãn "Dog"). Áp dụng cho ảnh có nhiều đối tượng.
- **Segmentation:** Trả lời câu hỏi **"Pixel nào thuộc về đối tượng nào?"**. Đây là tác vụ chi tiết nhất. Mô hình phân loại từng pixel trong ảnh. Kết quả là một **"mặt nạ" (mask) tô màu chính xác khu vực** của từng đối tượng. Segmentation lại được chia nhỏ thành
 - *Semantic Segmentation* (phân loại pixel theo lớp, không phân biệt các cá thể, ví dụ tất cả các con chó đều cùng một màu)
 - *Instance Segmentation* (phân biệt từng cá thể riêng biệt, ví dụ mỗi con chó là một màu khác nhau).

- **Quá trình huấn luyện mô hình:**

- Chúng ta cung cấp cho mạng neural rất nhiều ảnh đã được gán nhãn (labeled photos), ví dụ hàng ngàn ảnh mèo được gán nhãn "CAT" và hàng ngàn ảnh chó được gán nhãn "DOG".
- Mạng neural sẽ xử lý các ảnh này và dần dần học cách phân biệt giữa mèo và chó để tạo ra một "OUTPUT" (đầu ra) chính xác.
 - Ảnh đầu vào được chuyển đổi thành các **vector số** (pixel → đặc trưng → đặc trưng trừu tượng hơn qua từng lớp).
 - Mỗi lớp giúp trích xuất các đặc điểm từ thấp đến cao, chẳng hạn:
 - Lớp đầu: cạnh, góc, màu sắc.
 - Lớp giữa: hình dạng tai, mắt, mũi.
 - Lớp cuối: đặc trưng tổng hợp giúp phân biệt mèo và chó.
- Mô hình sau huấn luyện sẽ đưa ra dự đoán (output) cho ảnh mới.

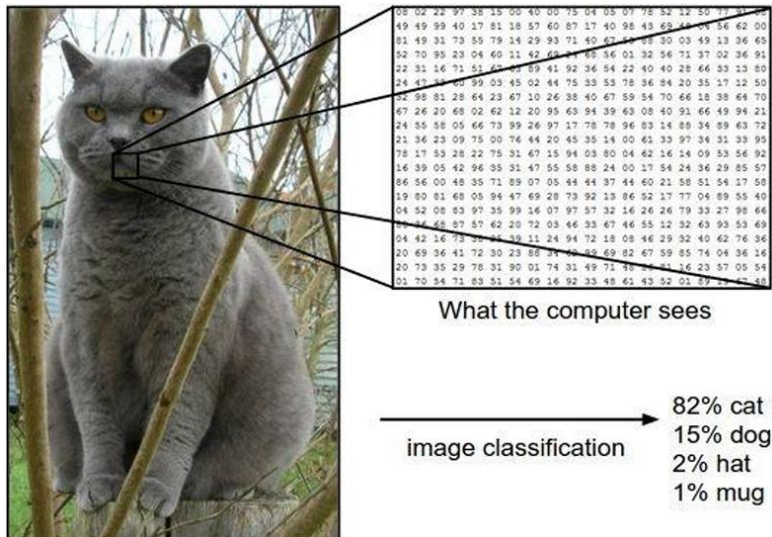
4.1.3. Neural Network và Deep Learning



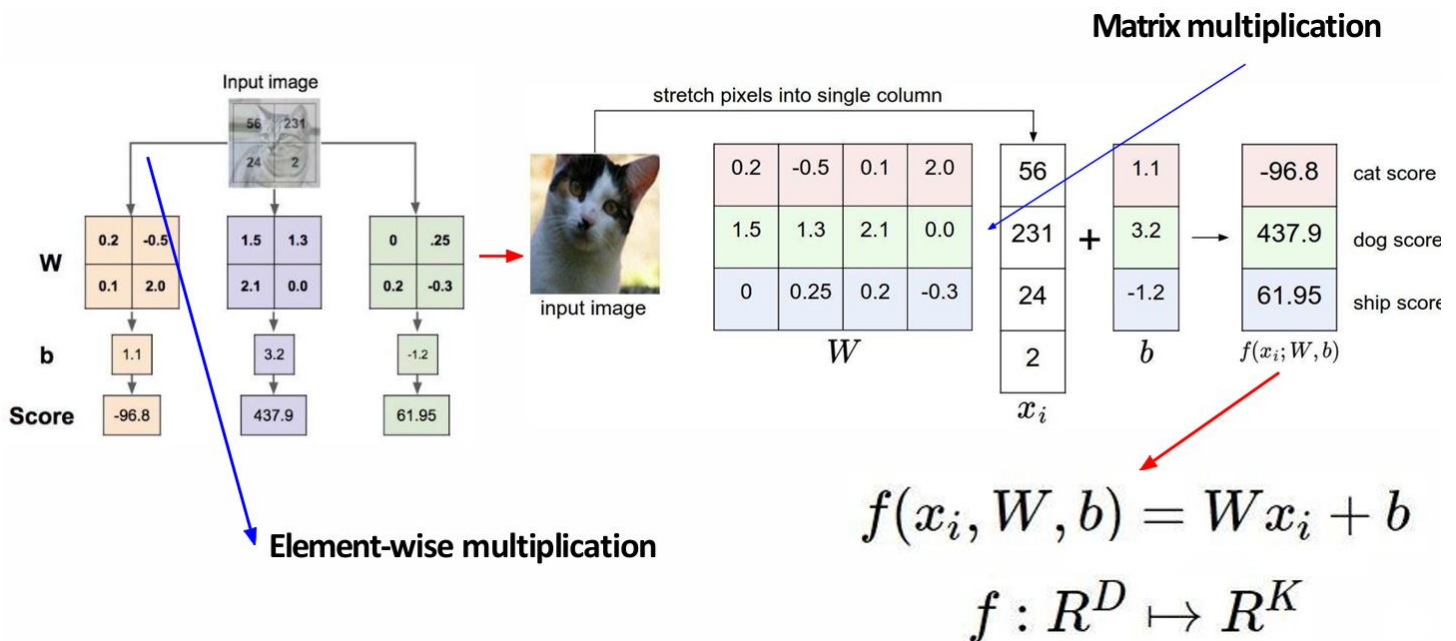
- **Machine Learning:** Là lĩnh vực lớn nhất, bao trùm tất cả.
- **Neural Networks:** Là một nhánh, một loại mô hình trong Machine Learning.
 - Vậy tầng ẩn của một neural network có cần tồn tại hay không? **Không nhất thiết phải có lớp ẩn, nhưng nếu không có lớp ẩn, mô hình rất hạn chế** — chỉ học được các quan hệ tuyến tính (linear).
- **Deep Learning:** Là một nhánh con của Neural Networks, sử dụng các mạng có kiến trúc sâu (nhiều lớp). "Deep" (sâu) ở đây chính là nói đến số lượng lớp. Có nhiều hơn một lớp ẩn → Học đặc trưng phức tạp, trừu tượng hơn.
- Mọi mô hình Deep Learning đều là Neural Network, và mọi Neural Network đều là một dạng của Machine Learning.

4.2. Linear Classifier – Bộ phân loại tuyến tính

- Đơn vị cơ bản nhất để xây dựng một mạng neural cho bài toán hình ảnh.



- Con người nhìn thấy một con mèo xám.
- Máy tính chỉ nhìn thấy một ma trận các con số.** Mỗi con số đại diện cho cường độ sáng của một pixel (ví dụ: từ 0 là đen đến 255 là trắng). Mô hình **"image classification"** sẽ nhận ma trận số này và đưa ra một danh sách xác suất cho các lớp có thể có (82% là mèo, 15% là chó, 2% là mũ, 1% là cốc).
- Mô hình trước khi đưa ra xác suất dự đoán (nhận dạng) cần được huấn luyện thông qua một bộ dữ liệu lớn chứa hình ảnh của nhiều lớp khác nhau, ví dụ: cat, dog, mug, hat.



- Ý tưởng của bộ phân loại:** Một ảnh ($\rightarrow$ được biến đổi thành vector) $\rightarrow$ nhân với ma trận trọng số $\rightarrow$ cộng bias $\rightarrow$ ra điểm số cho từng lớp $\rightarrow$ **Điểm cao nhất sẽ quyết định nhãn cho ảnh.**
- Có 2 cách để tính ra điểm số cho từng lớp:

- **Element-wise multiplication (Hadamard product)**

- Ảnh đầu vào được biến đổi thành ma trận X , thực hiện tính toán ban đầu với ma trận trọng số W , gồm 3 ma trận con, mỗi ma trận đại diện cho một lớp (Chó, mèo, tàu).
- Mỗi pixel trong ảnh đầu vào X tương ứng với một trọng số W trong lớp đó.
- Ta nhân **từng phần tử tương ứng**: $Z = W \odot X$ với W, X phải cùng kích cỡ. Kết quả Z là ma trận có cùng kích thước với W . (Không phải phép nhân ma trận thông thường)
- Ví dụ:

$$W = \begin{bmatrix} 0.2 & 0.1 \\ 0.3 & -0.5 \end{bmatrix}, X = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

$$Z = W \odot X = \begin{bmatrix} 0.2 \times 10 & 0.1 \times 20 \\ 0.3 \times 30 & -0.5 \times 40 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 9 & -20 \end{bmatrix}$$

- Sau khi nhân phần tử, ta **cộng dồn toàn bộ giá trị trong** Z , được kết quả s .
Với ví dụ trên: $s = 2 + 2 + 9 + (-20) = -7$
- Bias giúp mô hình điều chỉnh “ngưỡng quyết định” cho từng lớp: **score = s + b**
Ví dụ, nếu bias $b = 1.5$: $\text{score} = -7 + 1.5 = -5.5$

- **Matrix multiplication**

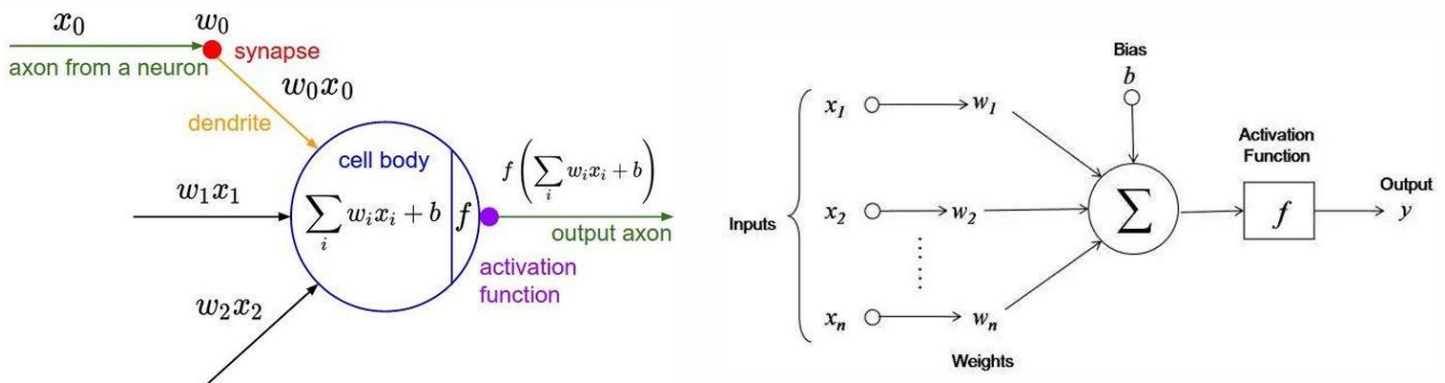
- Công thức cơ bản của bộ phân loại tuyến tính: $f(x_i, W, b) = W \cdot x_i + b$
 - x_i : vector ảnh đầu vào (sau khi làm phẳng – chuyển thành vector cột - vectorization)
 - Biến ảnh 2x2 $\rightarrow$ vector cột 4x1: $x_i = \begin{bmatrix} 56 \\ 231 \\ 24 \\ 2 \end{bmatrix}$
 - **b**: vector bias
 - Là một vector có kích thước bằng **số lớp (K)** $\rightarrow$ ở đây là 3x1 (cat, dog, ship).
 - Mỗi phần tử trong b là một giá trị điều chỉnh (offset) cho một lớp.
 - Công dụng: Giúp mô hình linh hoạt hơn, không bị ép đi qua gốc tọa độ (0,0).
 - Nguồn gốc của vector bias:
 - Học từ dữ liệu (training)
 - Khởi tạo ban đầu
 - Cập nhật trong quá trình train
 - **W**: ma trận trọng số (weight matrix)
 - Ma trận $K \times D$ (K = số lớp, D = số đặc trưng = số pixel sau làm phẳng).
 - Mỗi **hàng** trong W là một **bộ lọc (template)** cho **một lớp**.
 - Hàng 1: "mèo trông như thế nào" (theo pixel)
 - Hàng 2: "chó trông như thế nào"
 - Hàng 3: "tàu trông như thế nào"

- Nguồn gốc của ma trận trọng số:
 - Học từ dữ liệu (training)
 - Khởi tạo ban đầu
 - Cập nhật trong quá trình train
- f : hàm ánh xạ từ không gian ảnh (R^D) sang không gian điểm (R^K).
- Wx_i (matrix-vector multiplication): Thực hiện phép nhân ma trận thông thường.

• **Dog score cao nhất → model dự đoán ảnh là chó.**

- Mục tiêu của việc "huấn luyện" (training) chính là đi tìm ra các giá trị tốt nhất cho ma trận W và vector b để mô hình đưa ra dự đoán chính xác trên dữ liệu.
- Đầu ra của quá trình này là **con số** (ví dụ score trong ví dụ trên). Con số này sẽ được sử dụng để đưa vào **activation function** đề cập ở phần tiếp theo.

4.3. Từ Linear Classifier đến Neural Network



- Demo: [Simple forward pass](#)
- **Neuron = linear classifier + activation function**
- Lấy lại ví dụ ở 4.2

Thành phần	Ý nghĩa	Trong ví dụ ảnh mèo
x_0, x_1, x_2, x_n		4 pixel: 56, 231, 24, 2
w_0, w_1, w_2, w_n	4 trọng số	1 hàng trong ma trận W (ví dụ: [0.2, -0.5, 0.1, 2.0])
$w_i x_i$	Nhân từng cặp	$0.2 \times 56, -0.5 \times 231, \dots$
$\Sigma + b$	Cộng lại	= -96.8 (cho lớp mèo)
$f(\cdot)$	Hàm kích hoạt	ReLU → $\max(0, -96.8) = 0$
y	Output	0 → "không phải mèo"

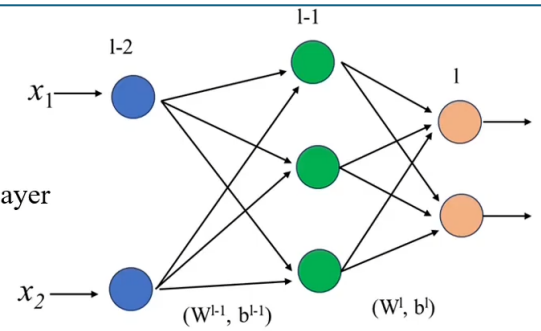
- Vậy đây là một neural. Nếu ta thực hiện tương tự cho 2 hàng còn lại trong ma trận W , dùng để dự đoán kết quả về chó & tàu, thì ta có thêm 2 neural nữa.
- Suy ra ta có **một mạng neural có 3 neural**.

Thành phần trong ví dụ	Tương ứng
Ảnh đầu vào (4 pixel)	Input layer
$W, b, \text{activation}$	Output layer
→ Không có Middle layer	

- **Output của neuron là xác suất hoặc giá trị phi tuyến.**

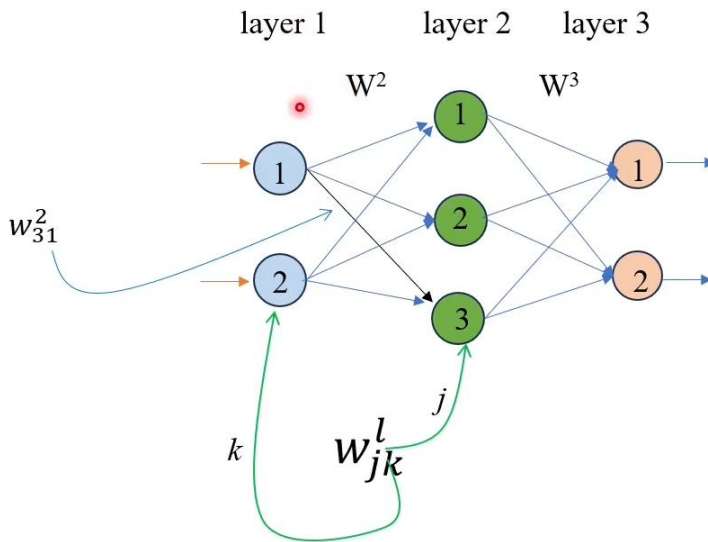
4.3.1. Một số quy ước ký hiệu

- Network has L layers from 1 to L . **Input layer is 1st layer**
- W^l is weights connecting nodes from the **($l-1$) layer to the l layer**
- w_{jk}^l is weight connecting **k^{th} node of the ($l-1$) layer to j^{th} node of the l layer**
- b^l is bias connecting from the **($l-1$) layer to the l layer**
- For input layer: $a_i^1 = x_i$



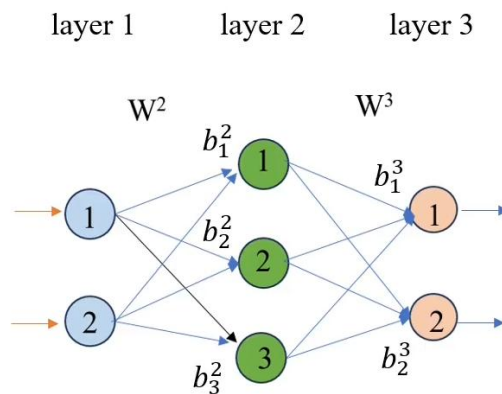
$$z_j^l = \sum_k w_{jk}^l \cdot a_k^{l-1} + b_j^l \quad \text{Weighted input of the } j^{\text{th}} \text{ node of the } l \text{ layer}$$

$$a_j^l = \sigma(z_j^l) \quad \text{Activation output/ Activation of the } j^{\text{th}} \text{ node of the } l \text{ layer}$$



$$W^2 = (w_{11}^2, w_{21}^2, w_{31}^2, w_{12}^2, w_{22}^2, w_{32}^2)$$

$$W^2 = \begin{bmatrix} w_{11}^2 & w_{12}^2 \\ w_{21}^2 & w_{22}^2 \\ w_{31}^2 & w_{32}^2 \end{bmatrix}$$

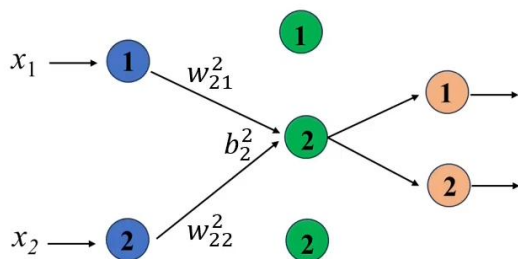
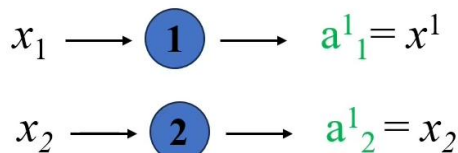


$$b^l = \begin{bmatrix} b_1^l \\ b_2^l \\ b_3^l \end{bmatrix}$$

b_k^l is bias of the k – neuron in the l – layer

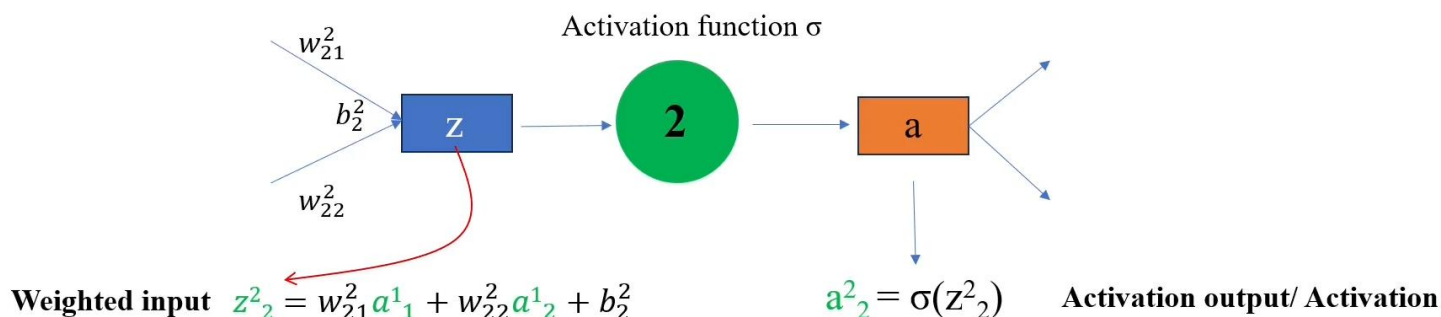
NOTE: We do not have W^1 and b^1 because this is input layer

Input layer: activation function is $f(x) = x$



$$z_j^l = \sum_k w_{jk}^l \cdot a_k^{l-1} + b_j^l$$

$$a_j^l = \sigma(z_j^l)$$



4.3.2. Activation function – Hàm kích hoạt

- Tại sao cần Activation Function?

Không có f	Có f
Mạng chỉ là tuyến tính → dù bao nhiêu tầng cũng = 1 phép tuyến tính - tương đương với một bộ phân loại tuyến tính duy nhất	Tạo phi tuyến tính → mô hình học được hàm phức tạp.
Không học được XOR, vòng tròn, v.v.	Học được bất kỳ hàm nào (theo định lý Universal Approximation)

Sigmoid	Tanh	ReLU	Leaky ReLU
$g(z) = \frac{1}{1 + e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$

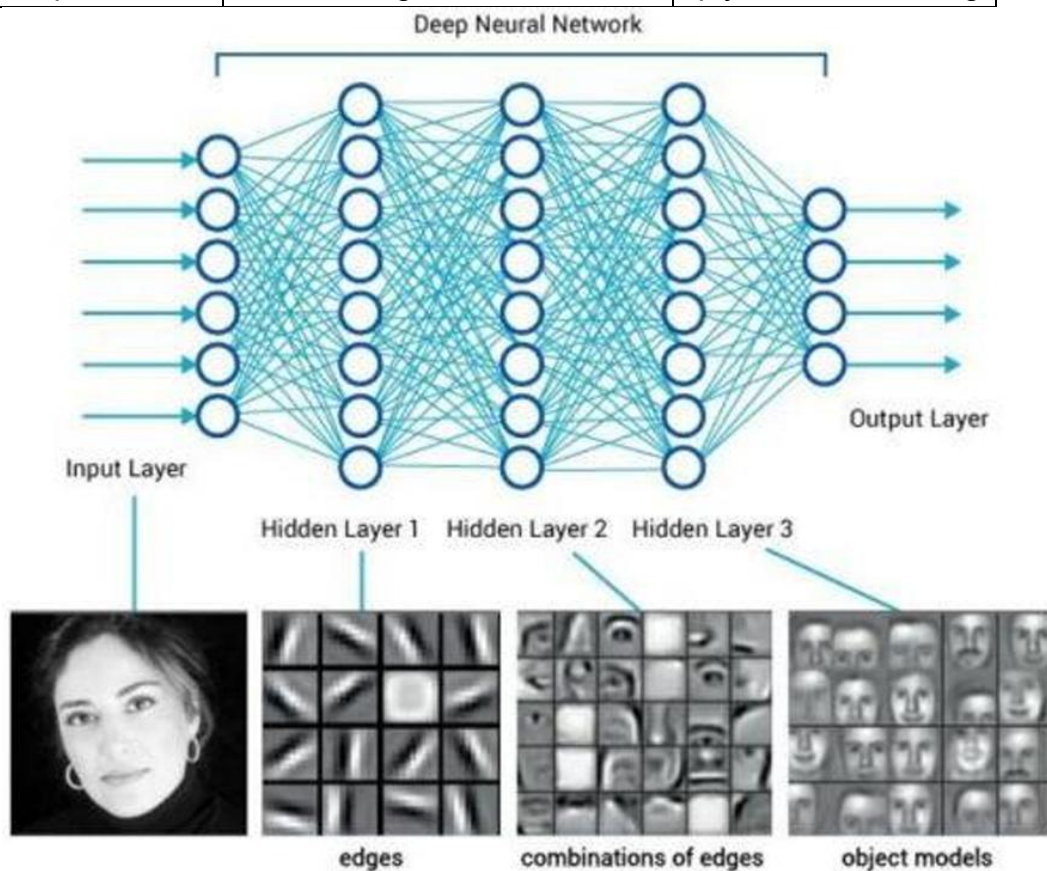
- **Sigmoid:**
 - Ép giá trị đầu ra vào khoảng (0, 1).
 - Thường dùng cho **bài toán nhị phân (binary classification)**. Ví dụ: Xác suất là mèo hay không phải mèo.
 - Không đối xứng quanh 0.
 - Từng rất phổ biến nhưng hiện nay ít dùng ở các lớp ẩn vì gây ra vấn đề Gradient nhỏ khi $|z|$ lớn $\rightarrow$ "vanishing gradient".
- **Tanh:**
 - Tương tự Sigmoid nhưng ép giá trị vào khoảng (-1, 1) & đối xứng quanh 0.
 - **Trung tâm hóa dữ liệu quanh 0**
 - Giảm bias khi truyền qua các lớp
 - Cũng bị **vanishing gradient** khi $|z|$ lớn
- **ReLU (Rectified Linear Unit):**
 - Là hàm phổ biến nhất trong **mạng học sâu (Deep Learning)**
 - Nếu đầu vào dương, giữ nguyên. Nếu âm, cho ra 0.
 - Rất hiệu quả về mặt tính toán và giúp giảm thiểu vấn đề vanishing gradient.
 - Khi $z < 0 \rightarrow$ neuron "chết" (Dead ReLU problem)
- **Leaky ReLU:**
 - **Một biến thể của ReLU**, nhưng khi $z < 0$ vẫn cho một giá trị nhỏ ϵz (ví dụ: $0.01z$).
 - Giúp *giải quyết vấn đề "dying ReLU"* (neuron chết).
 - Vẫn giữ tốc độ học tốt như ReLU
- **Lưu ý:**
 - **All nodes in a layer** should have the **same activation function**.
 - Activation function of **all hidden layers** are often the **same**.
 - Activation function of **output layer** can be **different** from those of **hidden layers**.

4.3.3. Neural Network Architecture

- Một mạng neural cơ bản có 3 loại lớp:
 - **Input Layer:** Nhận dữ liệu đầu vào (raw data).
 - **Số neuron = số đặc trưng (features) của dữ liệu.**
 - **Ví dụ:**
 - Nếu đầu vào là ảnh 28×28 pixel $\rightarrow$ có 784 giá trị $\rightarrow$ **784 neuron**.
 - trong ví dụ ảnh mèo là 4 neuron.
 - Nếu đầu vào là 4 đặc trưng của bộ dữ liệu Iris $\rightarrow$ **4 neuron**.
 - **Hidden Layer(s):** Các lớp nằm giữa.
 - Đây là nơi diễn ra các phép tính toán và **học hỏi** chính.
 - Nơi mà mạng học cách **biểu diễn (representation)** và **trích xuất đặc trưng** từ dữ liệu.
 - Học đặc trưng ẩn và mô hình hóa quan hệ phức tạp.

▪ Ví dụ:

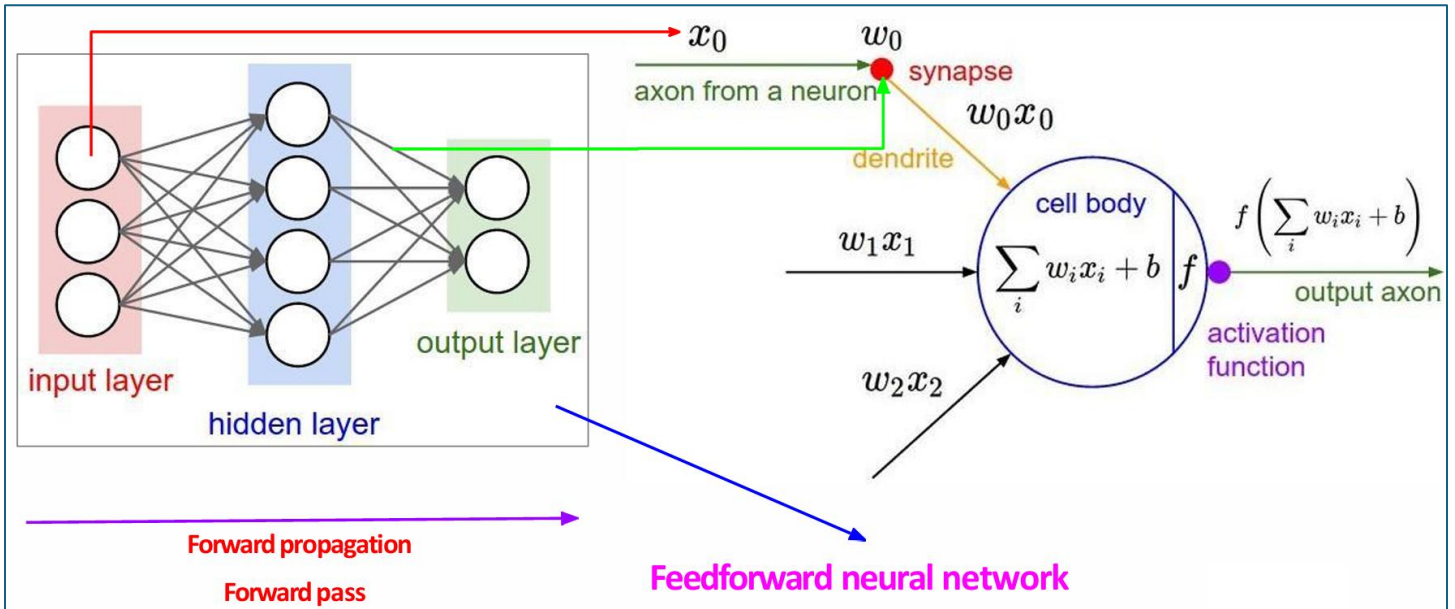
Tầng	Học được gì	Vai trò
Input	pixel của ảnh (giá trị sáng/tối)	dữ liệu thô
Hidden layer 1	học các đường, cạnh, góc	đặc trưng cơ bản
Hidden layer 2	học các tai, mắt, mũi	đặc trưng trung gian
Hidden layer 3	học toàn bộ khuôn mặt mèo	đặc trưng cao cấp
Output	mèo / không mèo	quyết định cuối cùng



- Mạng có nhiều hơn một lớp ẩn được gọi là mạng sâu (deep network).
- **Tối thiểu 1 neuron**, nhưng nếu chỉ có **1 neuron** thì khả năng biểu diễn rất kém (mạng gần như chỉ là tuyến tính).
- Thông thường, người ta chọn số neuron sao cho:

$$n_{\text{hidden}} \in [n_{\text{input}}, 2n_{\text{input}}]$$
 hoặc dùng quy tắc heuristic.
- Một số quy tắc kinh nghiệm:
 - $n_{\text{hidden}} = \frac{2}{3}(n_{\text{input}} + n_{\text{output}})$
 - Thử nghiệm và tinh chỉnh bằng cross-validation.
- Số neuron **không cần bằng nhau** giữa các lớp. Chẳng hạn:
 - Lớp ẩn thứ 1 có 8 neuron → học các **mối quan hệ phức tạp** giữa đặc trưng.
 - Lớp ẩn thứ 2 giảm còn 4 neuron → **nén lại thông tin quan trọng** trước khi ra kết quả.

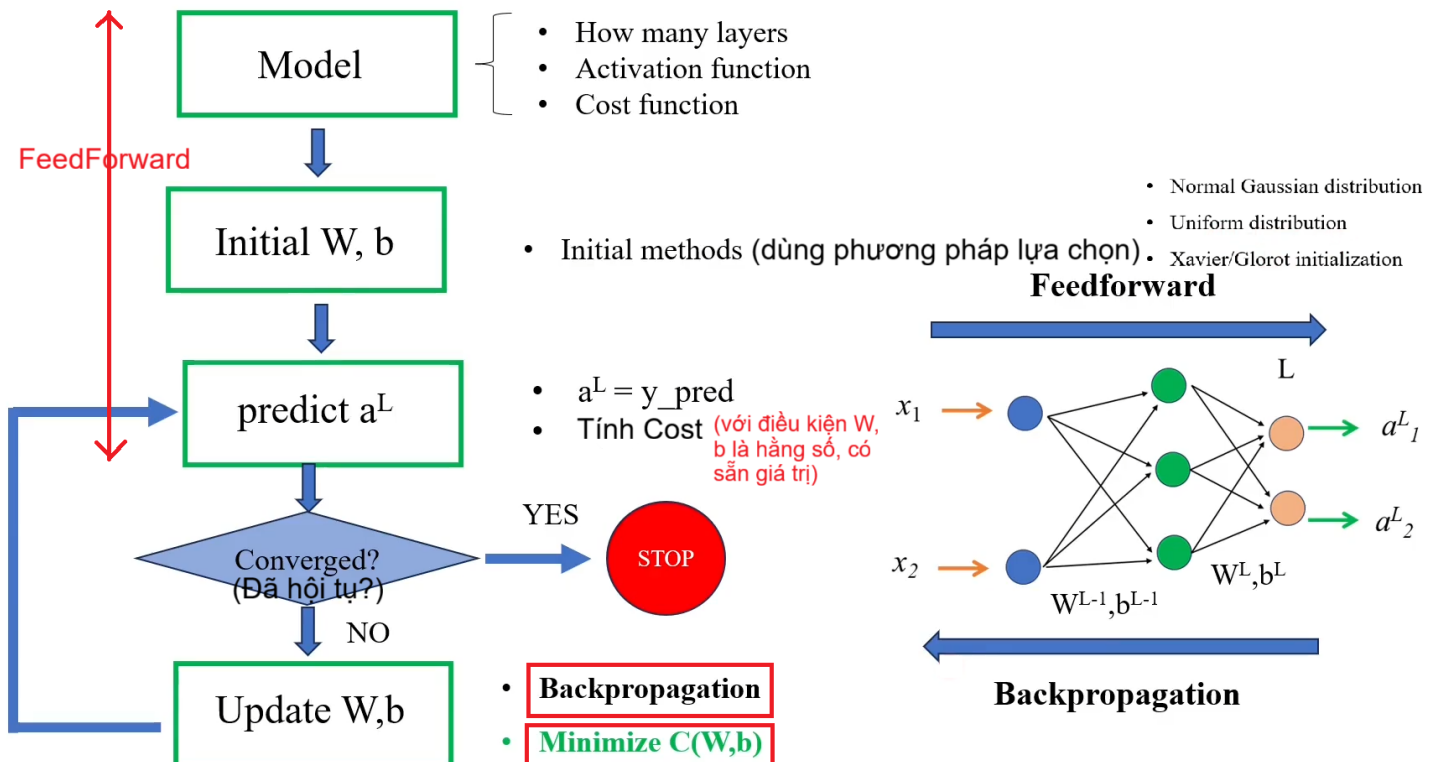
- **Output Layer:** Đưa ra kết quả cuối cùng.
 - Số neuron = số lớp nhân hoặc số giá trị đầu ra.
 - Bài toán phân loại 3 lớp → 3 neuron.
 - Bài toán hồi quy 1 giá trị → 1 neuron.



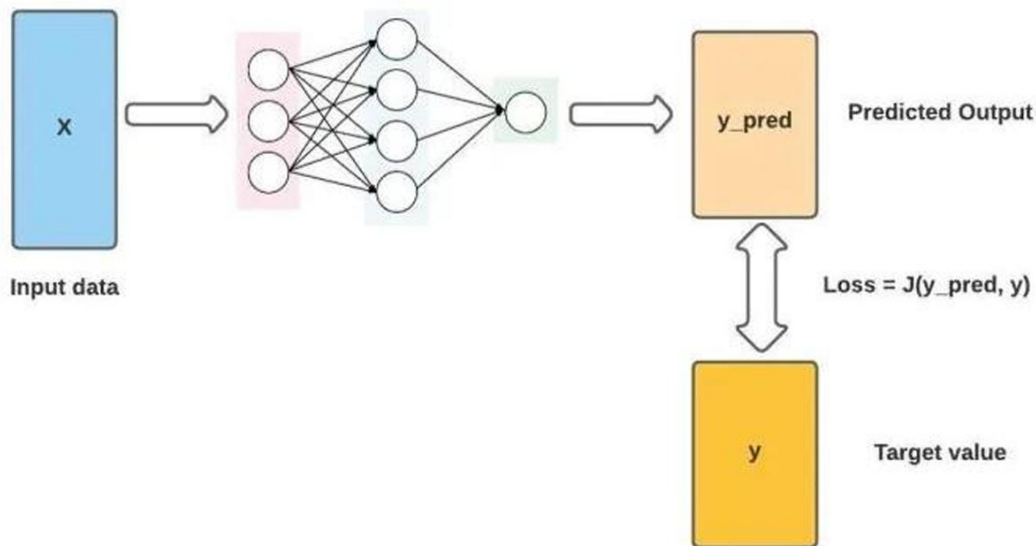
- **Feedforward Neural Network (FNN):** Là loại mạng nơ-ron nhân tạo cơ bản nhất, trong đó dòng dữ liệu chỉ đi theo một chiều — từ đầu vào → đầu ra (không có vòng lặp).
- Quá trình dữ liệu đi từ input layer qua hidden layer đến output layer được gọi là **Forward propagation** (lan truyền tiến/lan truyền thẳng) hay **Forward pass**.

4.4. Huấn luyện Mạng Neural

4.1.1. Quy trình huấn luyện



4.4.2. Loss Function (Hàm mất mát)



- Mô hình nhận **Input data (x)** và cho ra **Predicted Output (y_pred)**.
- Hàm mất mát so sánh **y_pred** với **Target value (y)** (nhấn đúng) để tính ra một con số Loss.
- **Loss càng cao, mô hình dự đoán càng tệ.**
- **Hàm mất mát (Loss Function)** dùng để đo **độ lệch (sai khác)** giữa **giá trị dự đoán** của mô hình $\hat{y}$ và **giá trị thực tế mong muốn y** .

- **Các loại hàm mất mát:**

Regression Loss Functions (Hàm mất mát hồi quy)

- Dùng khi dự đoán một giá trị liên tục (như giá nhà, nhiệt độ).
- Mục tiêu: Mô hình dự đoán ra giá trị **gần đúng nhất với giá trị thực tế**.
- **L1 Loss (Absolute Loss):** $L_1(y, \hat{y}) = |y - \hat{y}|$ → Tính trên một điểm (mẫu) dữ liệu

- **Mean Absolute Error (MAE)**

$$MAE = \frac{1}{n} \sum_{i=1}^n |y^{(i)} - \hat{y}^{(i)}|$$

- **n**: Tổng số mẫu trong tập dữ liệu
 - $y^{(i)}$: Giá trị thực tế (ground truth) của mẫu thứ i
 - $\hat{y}^{(i)}$: Giá trị dự đoán của model cho mẫu thứ i
 - Tính trên toàn bộ tập dữ liệu
 - **Ý nghĩa**: Tính **trung bình độ chênh lệch tuyệt đối** giữa dự đoán và giá trị thật.
 - **Ưu điểm**: Dễ hiểu, ít bị ảnh hưởng bởi giá trị ngoại lai (outlier) vì nó không bình phương sai số. Ví dụ, nếu hầu hết các dự đoán sai 1-2 đơn vị, nhưng có một dự đoán sai tới 50 đơn vị, MAE sẽ không bị tăng vọt như MSE.
 - **Nhược điểm**: Không trơn (non-differentiable) tại điểm 0 → khó tối ưu bằng gradient descent.
 - **Ví dụ**: Dự đoán giá nhà: sai trung bình 10 triệu là MAE = 10 triệu.

- **L2 Loss (Squared Loss):** $L_2(y, \hat{y}) = (y - \hat{y})^2$ → Tính trên một điểm (mẫu) dữ liệu

- **Mean Squared Error (MSE)**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

- Tính trên toàn bộ tập dữ liệu
- **Ý nghĩa:** Bình phương sai số, rồi lấy trung bình.
- **Ưu điểm:** Đây là hàm mất mát phổ biến nhất cho bài toán hồi quy. Việc bình phương sai số có hai tác dụng chính:
 - Luôn cho ra giá trị dương.
 - **Khuếch đại các lỗi lớn.** Nếu mô hình dự đoán sai 10 đơn vị, sai số bình phương là 100. Nếu sai 2 đơn vị, sai số bình phương chỉ là 4. Điều này "trừng phạt" các lỗi lớn nặng hơn nhiều so với lỗi nhỏ, buộc mô hình phải cố gắng giảm thiểu các lỗi này.
- **Nhược điểm:** Nhạy với outlier. Một dự đoán cực kỳ tệ có thể làm giá trị MSE tăng vọt và ảnh hưởng lớn đến quá trình huấn luyện.
 - **Ví dụ:** Nếu dự đoán sai nhiều ở 1 điểm, MSE tăng rất mạnh → buộc mô hình "học kỹ" hơn.

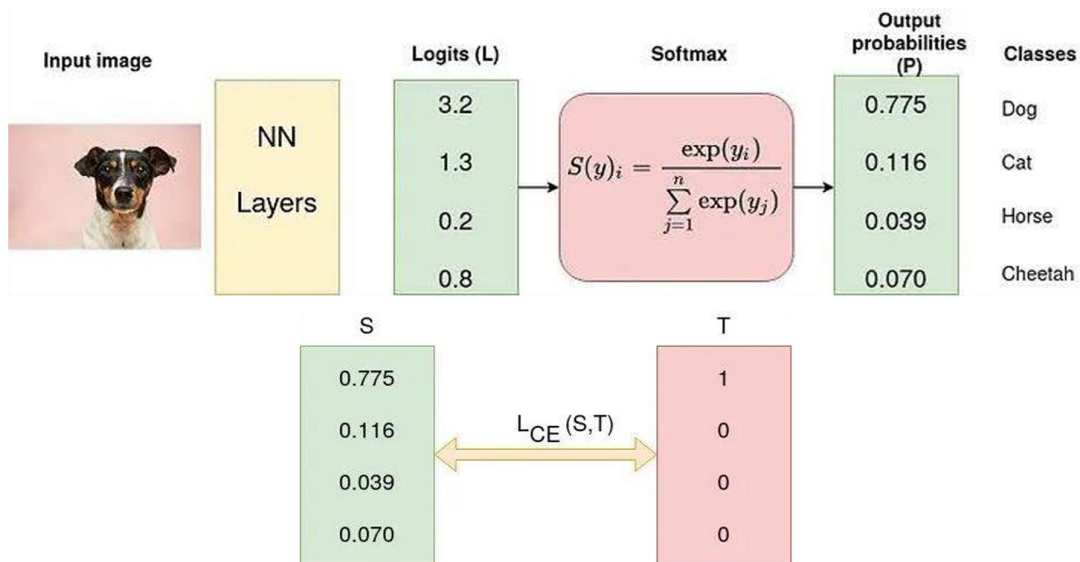
- **Huber Loss (Smooth L1 Loss)**

$$Huber = \frac{1}{n} \sum_{i=1}^n \begin{cases} \frac{1}{2} (y_i - \hat{y}_i)^2, & \text{nếu } |y_i - \hat{y}_i| \leq \delta \\ \delta (|y_i - \hat{y}_i| - \frac{1}{2} \delta), & \text{nếu } |y_i - \hat{y}_i| > \delta \end{cases}$$

- δ (delta): Là một **siêu tham số** (hyperparameter) do chúng ta tự chọn. Nó là một ngưỡng để quyết định khi nào dùng công thức của MSE, khi nào dùng của MAE.
- **Ý nghĩa:** Kết hợp MSE và MAE:
 - Với sai số nhỏ → giống MSE (bình phương).
 - Với sai số lớn → giống MAE (tuyến tính, tránh bị ảnh hưởng outlier).
- **Ưu điểm:** Mượt mà, cân bằng giữa MSE và MAE.
- **Thường dùng trong:** Dự đoán giá trị có thể có outlier (như giá cổ phiếu, nhiệt độ...).
- Đây là một lựa chọn tốt khi bạn muốn một hàm mất mát ổn định, vừa có thể xử lý các lỗi lớn một cách nhẹ nhàng, vừa tinh chỉnh tốt khi lỗi nhỏ.

Classification Loss Functions (phân loại rời rạc)

- Dùng khi dự đoán một **lớp** (như mèo, chó) (phân loại rời rạc).
- Mục tiêu: Mô hình dự đoán **xác suất đúng nhất** cho từng lớp.



- **NN Layers:** Mạng Neural xử lý ảnh đầu vào và cho ra một vector gọi là **Logits (L)**. Đây là các điểm số "thô", có thể nhận bất kỳ giá trị thực nào (âm, dương, lớn, nhỏ).
- **Softmax:** Hàm Softmax được áp dụng lên vector Logits để chuyển nó thành một **vector xác suất (P)**. Mỗi giá trị trong vector P nằm trong khoảng và tổng tất cả các giá trị bằng 1.
 - Trong đó y_i là logit thứ i .
 - Kết quả P: [0.775, 0.116, 0.039, 0.070]. Ta có thể diễn giải là: mô hình dự đoán 77.5% là Chó, 11.6% là Mèo, 3.9% là Ngựa, và 7.0% là Báo.
- **Tính Loss:** Cross-Entropy Loss sẽ so sánh vector xác suất dự đoán S (hoặc P) với vector thực tế T.
 - T là một **one-hot vector**. Vì ảnh thực tế là chó (lớp đầu tiên), vector T sẽ là [1, 0, 0, 0].
- **Cross-Entropy Loss (Hàm mất mát Entropy chéo)**

$$L_{CE} = - \sum_{i=1}^n t_i \log(p_i)$$

- t_i : Nhãn thực tế (one-hot vector, ví dụ: [1, 0, 0, 0])
- p_i : Xác suất mô hình dự đoán sau Softmax (ví dụ: [0.775, 0.116, 0.039, 0.07])
- $$L_{CE} = - [(t_{dog} * \log(p_{dog})) + (t_{cat} * \log(p_{cat})) + (t_{horse} * \log(p_{horse})) + (t_{cheetah} * \log(p_{cheetah}))]$$

$$L_{CE} = - [(1 * \log(0.775)) + (0 * \log(0.116)) + (0 * \log(0.039)) + (0 * \log(0.070))]$$

$$L_{CE} = - [\log(0.775) + 0 + 0 + 0] = -\log(0.775) \approx 0.25$$
- **Mục tiêu của mô hình là làm cho L_{CE} nhỏ nhất có thể.**
- Nếu mô hình dự đoán rất tự tin và chính xác, $p_{dog} \approx 1$. Khi đó, $\log(p_{dog}) \approx 0$, và $L_{CE} \approx 0$ (rất nhỏ).
- Nếu mô hình dự đoán rất sai, ví dụ $p_{dog} \approx 0.01$. Khi đó, $\log(p_{dog})$ là một số rất âm, và $L_{CE} = -\log(p_{dog})$ sẽ là một số rất lớn.
- **Công dụng của Cross-Entropy Loss**
 - Được dùng phổ biến trong **classification** (nhị phân hoặc đa lớp).
 - Dựa trên **khoảng cách giữa phân phối xác suất thực tế và dự đoán**.
 - Kết hợp **Softmax + CrossEntropy** giúp mô hình học xác suất "mềm" (soft probabilities).

- **Loss Function** thực chất dùng để tính độ lệch giữa giá trị đầu ra gốc và giá trị đầu ra dự đoán của **từng mẫu dữ liệu**.
- **Cost function** dùng để tính **trung bình độ lệch trên toàn tập dữ liệu**. Nhưng trong thực tế, sách vở và slide **thường gộp chung** và gọi luôn là *Loss function*, dù nó đang là **loss tính trên cả dataset**.

4.4.3. Gradient & Gradient Descent

4.4.3.1. Gradient là gì?

- Đạo hàm cho chúng ta biết **tốc độ thay đổi của hàm dựa trên một biến số duy nhất (tại một thời điểm nào đó)** thường được đặt là **x**.
 - Ví dụ, **dF/dx** cho chúng ta biết hàm **F** thay đổi bao nhiêu khi **x** thay đổi 1 đơn vị.
 - Đối với một hàm có nhiều biến, chẳng hạn như **x** và **y**, nó sẽ có nhiều đạo hàm: các giá trị **dF/dx** và **dF/dy** cho ta biết giá trị của hàm sẽ thay đổi thế nào nếu ta thay đổi một biến số (với giả định các biến số khác không thay đổi, *hay coi phần còn lại không chứa biến đang xét là các hằng số*).
 - **Kí hiệu chuẩn về đạo hàm**
 - Khi hàm $f(x)$ là hàm 1 biến x , ví dụ: $f(x) = 2 * x + 1$. Đạo hàm của f đối với biến x kí hiệu là $\frac{df}{dx}$.
 - Khi hàm $f(x, y)$ là hàm nhiều biến, ví dụ $f(x, y) = x^2 + y^2$. Đạo hàm f với biến x kí hiệu là $\frac{\partial f}{\partial x}$.
- Chúng ta có thể biểu diễn các tỷ lệ này trong một vector nhiều thành phần, với một thành phần là một đạo hàm. Do đó, **một hàm có n biến sẽ có một **gradient** với n thành phần là các đạo hàm riêng theo n biến độc lập của hàm**.

- Chẳng hạn: Gradient của hàm $f(x, y)$ được ký hiệu là: **$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$** . Đây là **vector gradient**,

gồm các **đạo hàm riêng** của f theo từng biến x và y .

- Nó có thể viết lại dưới dạng vector hàng: $\nabla f(x, y) = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]^T$
- Hoặc dưới dạng **tổ hợp vector đơn vị**: $\nabla f = \frac{\partial f}{\partial x} \mathbf{i} + \frac{\partial f}{\partial y} \mathbf{j}$ (trong không gian 2 chiều, **i** và **j** là các vector đơn vị theo trục x và y).
- Với hàm ba biến: $\nabla f(x, y, z) = \langle f_x, f_y, f_z \rangle = \frac{\partial f}{\partial x} \mathbf{i} + \frac{\partial f}{\partial y} \mathbf{j} + \frac{\partial f}{\partial z} \mathbf{k}$ (**k** là vector đơn vị theo trục z).

Handwritten mathematical derivation on a blackboard:

$$f(x, y) = x^2 \sin(y)$$

$$\frac{\partial f}{\partial x} = 2x \sin(y)$$

$$\frac{\partial f}{\partial y} = x^2 \cos(y)$$

$$\nabla f(x, y) = \begin{bmatrix} 2x \sin(y) \\ x^2 \cos(y) \end{bmatrix}$$

- Gradient của một hàm là **véc tơ** biểu diễn **tốc độ thay đổi** của hàm theo từng hướng.
- **Hướng của vector gradient**: chỉ hướng mà hàm tăng nhanh nhất.
- **Độ lớn của vector gradient** (magnitude) biểu diễn **tốc độ thay đổi lớn nhất** của hàm tại điểm đó.
- Ví dụ minh họa

$$f = 3x^2 - 2xy + 5y^2$$

$$\frac{\partial f}{\partial x} = 6x - 2y; \quad \frac{\partial f}{\partial y} = -2x + 10y$$

$$\nabla f = \begin{bmatrix} 6x - 2y \\ -2x + 10y \end{bmatrix} \rightarrow \nabla f(1,2) = \begin{bmatrix} 6 \cdot 1 - 2 \cdot 2 \\ -2 \cdot 1 + 10 \cdot 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 18 \end{bmatrix}$$

- Gradient chỉ ra **hướng và tốc độ thay đổi nhanh nhất** của hàm **tại một điểm** cụ thể. Nếu bạn đánh giá gradient tại điểm (1,2), ta có:

$$\nabla f(1,2) = \begin{bmatrix} 6(1) - 2(2) \\ -2(1) + 10(2) \end{bmatrix} = \begin{bmatrix} 6 - 4 \\ -2 + 20 \end{bmatrix} = \begin{bmatrix} 2 \\ 18 \end{bmatrix}$$

- Vậy gradient tại điểm (1,2) là $\begin{bmatrix} 2 \\ 18 \end{bmatrix}$. Điều này có nghĩa là **tại điểm (1, 2), hàm $f(x, y)$ tăng nhanh nhất theo hướng của vector $\begin{bmatrix} 2 \\ 18 \end{bmatrix}$** , với tốc độ thay đổi là tỷ lệ với độ dài của vector gradient này.

- Giống như đạo hàm thông thường, **gradient chỉ ra hướng thay đổi mạnh nhất của hàm**.
- Ứng dụng phổ biến của gradient là **tìm kiếm các giá trị cực đại và cực tiểu của hàm**. Đây là nhiệm vụ quan trọng để tối ưu các trọng số trong mạng Deep learning.
- Trong một hàm đơn biến, để thay đổi giá trị của hàm, chúng ta chỉ có **hai hướng “tiến” và “lùi”** dựa trên sự thay đổi trên trục x.
 - Đối với hàm đa biến, mọi việc không đơn giản như vậy. Với hàm hai biến, chúng ta có thể tưởng tượng đó là vô số các hướng trong một mặt phẳng, với hàm ba biến, đó có thể là bất kỳ hướng nào trong không gian 3 chiều.
 - Với hàm 100 biến (như đầu vào của mạng nơ ron sử dụng word embedding 100 chiều), nó vượt ra khỏi sức tưởng tượng của chúng ta. *Mặc dù vậy, gradient là công cụ toán học tuyệt vời để ta có thể khảo sát sự biến thiên của những hàm này.*
- Đối với hàm một biến, ta có thể dùng đạo hàm, lập bảng biến thiên theo phương pháp khi học phổ thông để tìm cực tiểu (cực đại) của nó.
- Tuy nhiên, đối với hàm có nhiều biến, ta không thể sử dụng cách trên. Điều tốt hơn có thể làm là dùng gradient. Cụ thể là gradient descent, dùng để tìm cực tiểu, là cơ sở để tối thiểu hóa hàm mất mát.

4.4.3.2. Gradient Descent

- **Gradient Descent** là **thuật toán tối ưu** được dùng để **tìm giá trị nhỏ nhất** (minimum) của một hàm số.

- Ý tưởng:

- Gradient (∇f) chỉ hướng **tăng nhanh nhất** của hàm $f(x)$.
- Muốn **giảm giá trị của hàm**, ta đi **ngược hướng gradient**.

- Tham khảo: <https://www.youtube.com/watch?v=c7kGj3wyVgg>

- Giả sử ta muốn tìm giá trị nhỏ nhất của hàm $f(x)$, ta bắt đầu từ một điểm ban đầu x_0 và lặp lại:

$$x_{new} = x_{old} - \eta \cdot \nabla f(x_{old})$$

- Trong đó:

- η (eta) là **learning rate** – tốc độ học, hay **bước di chuyển**.
- $\nabla f(x_{old})$ là **gradient** tại vị trí hiện tại.
- Dấu **trừ (-)** vì ta di chuyển **ngược hướng tăng**.

- **Ví dụ minh họa đơn giản**

- Giả sử ta có hàm: $f(x) = x^2$
- Khi đó: $\frac{df}{dx} = 2x$
- Công thức cập nhật: $x_{new} = x_{old} - \eta(2x_{old})$
- Giả sử: $x_0 = 4, \eta = 0.1$
- Ta có:

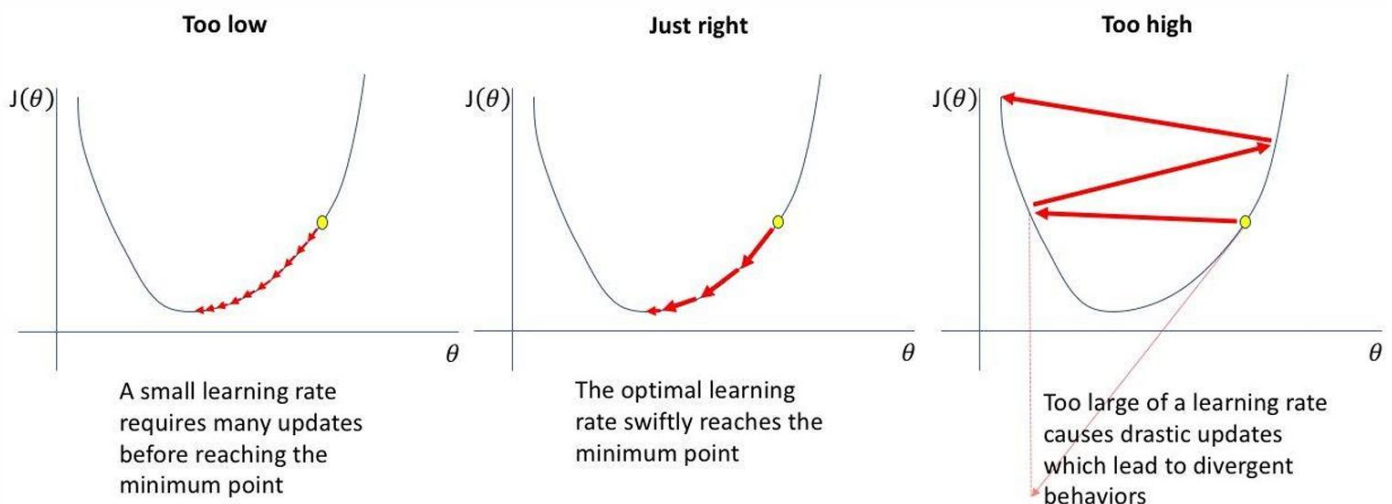
Bước	x	Gradient $2x$	Cập nhật $x - 0.1(2x)$
0	4.0	8	$4 - 0.1 \times 8 = 3.2$
1	3.2	6.4	$3.2 - 0.1 \times 6.4 = 2.56$
2	2.56	5.12	$2.56 - 0.1 \times 5.12 = 2.048$
...	...	...	...

- Sau nhiều bước, x dần tiến về 0 — là **điểm cực tiểu** của $f(x) = x^2$.

- Trong **Machine Learning**, ta thường cần **giảm sai số (loss function)** giữa dự đoán và dữ liệu thật.

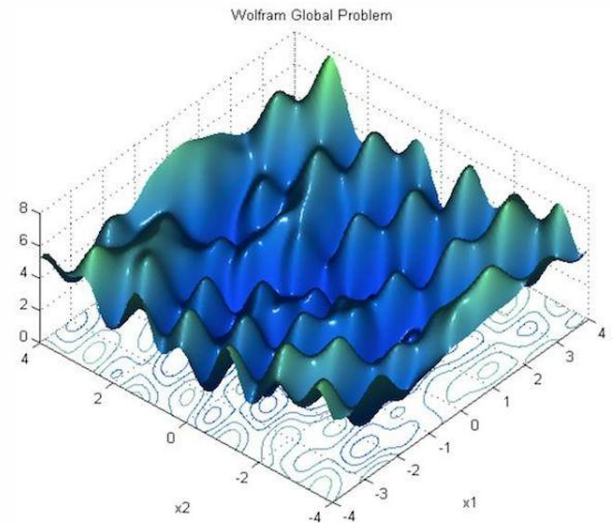
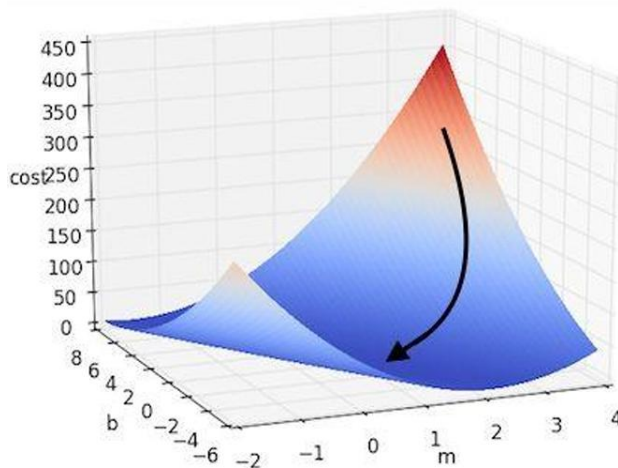
- Hàm mất mát (loss) đóng vai trò như $f(x)$
- Các tham số (weights) đóng vai trò như x
- **Gradient Descent** được dùng để **tối ưu hóa tham số** sao cho sai số nhỏ nhất.

- **Vai trò của Learning Rate (η):**

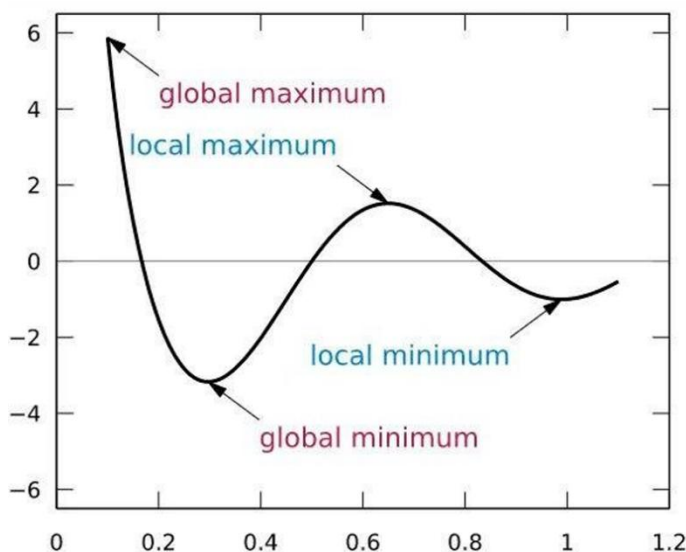


- **Too low (Quá thấp):** Các bước đi quá nhỏ. Mô hình học rất chậm và có thể mất rất nhiều thời gian để hội tụ.
- **Too high (Quá cao):** Các bước đi quá lớn. Mô hình có thể "nhảy" (băng) qua điểm đáy và không bao giờ tìm thấy nó. Nó sẽ dao động quanh điểm tối ưu hoặc thậm chí phân kỳ (diverge), tức là Loss ngày càng tăng.
- **Just right (Vừa phải):** Kích thước bước đi tối ưu, giúp mô hình hội tụ nhanh và ổn định.
- Việc chọn Learning Rate là một trong những công việc quan trọng và khó khăn nhất khi huấn luyện mô hình. Các kỹ sư thường phải thử nghiệm nhiều giá trị khác nhau. Có những kỹ thuật nâng cao gọi là "Learning Rate Schedulers" để tự động điều chỉnh η trong quá trình huấn luyện (ví dụ: bắt đầu với η lớn và giảm dần theo thời gian).

• Gradient Descent in Linear Classifier vs Neural network



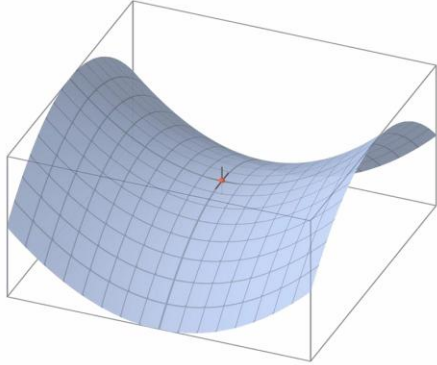
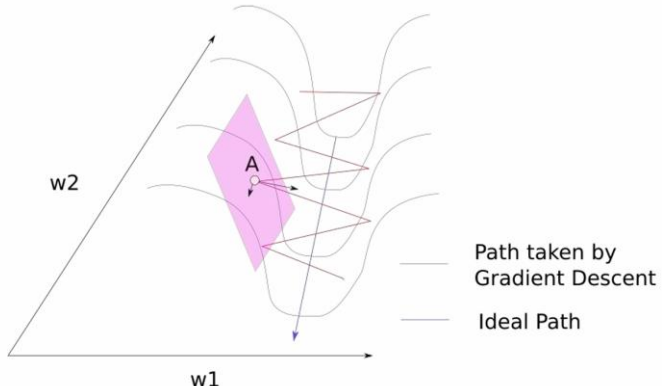
- **Linear Classifier (Bên trái):** Bề mặt hàm mất mát có dạng một cái bát (hình paraboloid).



Dạng này được gọi là **hàm lồi (convex function)**. Nó chỉ có **một điểm đáy duy nhất** là điểm tối ưu toàn cục (global minimum). Với bề mặt này, Gradient Descent được **đảm bảo sẽ tìm thấy điểm đáy, bất kể bạn bắt đầu từ đâu**.

- **Neural Network (Bên phải):** Bề mặt hàm mất mát cực kỳ phức tạp, lồi lõm, có vô số "ngọn đồi" và "thung lũng". Dạng này được gọi là **hàm không lồi (non-convex function)**. Khi áp dụng Gradient Descent trên bề mặt này, nó có thể dễ dàng bị **"kẹt" trong một thung lũng cận (local minimum)**. Tức là, **nó tìm thấy một điểm đáy, nhưng không phải là điểm đáy sâu nhất**. Đây là một trong những thách thức lớn nhất khi huấn luyện mạng neural sâu.

- Một số thách thức khác của **Gradient Descent**

	
Saddle points (Điểm yên ngựa): ➤ Đây là một điểm còn tệ hơn cả local minimum. ➤ Tại điểm này, gradient bằng 0, nhưng nó không phải là điểm đáy. ➤ Nó là điểm đáy theo một chiều nhưng lại là điểm đỉnh theo chiều khác (giống như yên ngựa). ➤ Các thuật toán GD cơ bản có thể bị "bất động" tại đây.	Ravines (Khe núi hẹp): ➤ Tưởng tượng một thung lũng rất hẹp và dốc. Thành thung lũng rất dốc, nhưng đáy thung lũng lại thoải. ➤ Gradient sẽ luôn chỉ vào thành thung lũng (hướng dốc nhất), khiến cho thuật toán cứ dao động qua lại giữa hai bên thành mà tiến rất chậm dọc theo đáy thung lũng (Ideal Path). ➤ Đây là một nguyên nhân phổ biến làm chậm quá trình hội tụ.

❖ Hội tụ trong Gradient Descent ○ Trong thuật toán Gradient Descent, hội tụ xảy ra khi: ○ Các giá trị của x (hay tham số) không thay đổi nhiều sau mỗi bước cập nhật. ○ Tức là thuật toán đã tìm được điểm cực tiểu (hoặc gần điểm cực tiểu) của hàm số. ❖ Điều kiện để Gradient Descent hội tụ: ○ Hàm phải có cực tiểu (minimum) và được khả vi (để tính toán đạo hàm). ○ Learning rate (η) phải đủ nhỏ để đảm bảo thuật toán không nhảy quá xa, tránh việc bỏ qua cực tiểu. ○ Nếu bước nhảy quá lớn (learning rate quá cao), thuật toán có thể không hội tụ, mà thay vào đó sẽ dao động hoặc phát tán ra ngoài. ❖ Giả sử ta có hàm $f(x) = x^2$ và ta áp dụng Gradient Descent để tìm cực tiểu của hàm này: ○ Ban đầu, ta chọn $x_0 = 4$. ○ Sau một số bước cập nhật: $x_1 = 3.2, x_2 = 2.56, x_3 = 2.048, \dots$ ○ Các giá trị x_n dần tiến gần về 0, và sự thay đổi giữa các bước trở nên ngày càng nhỏ hơn. ○ Khi sự thay đổi giữa các bước quá nhỏ, ta nói rằng thuật toán hội tụ tại giá trị $x = 0$. ❖ Trong học máy, thuật ngữ "hội tụ" có thể được dùng trong nhiều tình huống khác nhau, ví dụ: ○ Hội tụ của hàm mất mát (Loss function): Khi hàm mất mát không còn giảm nhiều sau mỗi vòng huấn luyện, ta nói rằng hàm mất mát đã hội tụ. ○ Hội tụ của mô hình: Mô hình huấn luyện đạt đến trạng thái không thay đổi đáng kể trong quá trình tối ưu hóa (tức là không thay đổi tham số nhiều nữa).
--

- 3 phiên bản của Gradient Descent

- Batch Gradient Descent (Toàn Lô):

- **Cách hoạt động:** Tất cả datapoints được đưa vào mô hình cùng 1 lúc để tính gradient. Tức là, mô hình phải xem qua toàn bộ tập dữ liệu huấn luyện, tính toán sai số trên tất cả chúng, rồi mới tính gradient trung bình và cập nhật trọng số một lần.
 - **Đường đi:** Rất mượt mà và đi thẳng đến điểm tối ưu.
 - **Nhược điểm:** Cực kỳ chậm và tốn bộ nhớ với các bộ dữ liệu lớn (hàng triệu ảnh). Không khả thi trong thực tế.

- Stochastic Gradient Descent (SGD - Ngẫu nhiên):

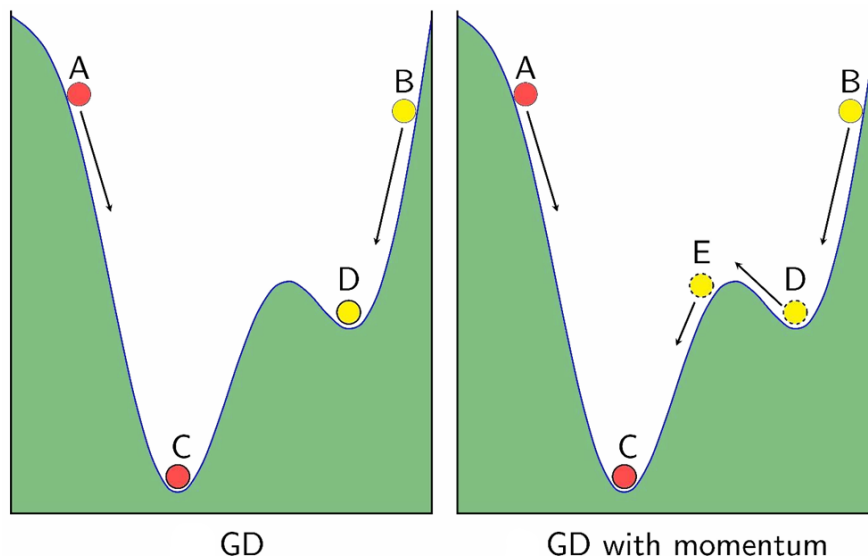
- **Cách hoạt động:** Từng datapoint một sẽ được đưa vào mô hình để tính gradient. Mô hình chỉ lấy một điểm dữ liệu ngẫu nhiên, tính loss và gradient, rồi cập nhật trọng số ngay lập tức.
 - **Đường đi:** Rất "nhiều" và đi zic-zac lung tung như người say rượu. Tuy nhiên, xu hướng chung vẫn là đi về phía điểm tối ưu.
 - **Ưu điểm:** Cập nhật rất nhanh. Sự "nhiều" đôi khi lại tốt, giúp nó có khả năng "nhảy" ra khỏi các điểm local minima cạn.

- (Stochastic) Mini-batch Gradient Descent (Lô nhỏ):

- **Cách hoạt động:** N datapoints sẽ được đưa vào mô hình cùng lúc để tính gradient. Đây là sự thỏa hiệp hoàn hảo. Thay vì dùng cả tập dữ liệu hoặc 1 điểm, ta dùng một "lô" nhỏ (ví dụ 32, 64, 128 ảnh).
 - **Đường đi:** Ít nhiễu hơn SGD nhưng vẫn có sự ngẫu nhiên, và hiệu quả tính toán cao hơn nhiều so với Batch GD.
 - **Mini-batch Gradient Descent là phương pháp tiêu chuẩn được sử dụng trong hầu hết các ứng dụng Deep Learning hiện nay.** Khi người ta nói "SGD", thường họ đang ngụ ý đến "Mini-batch SGD".

4.4.3.3. Momentum (Đà)

- một cải tiến cho SGD để giải quyết các thách thức của GD.



- Tưởng tượng một quả bóng lăn xuống dốc thay vì một người đi bộ. Quả bóng có "đà" (momentum).

- Khi đi qua một khe núi hẹp (ravine), đà sẽ giúp "làm mịn" các dao động, giúp quả bóng lăn nhanh hơn dọc theo đáy khe.
- Khi gặp một chỗ lồi lõm nhỏ (local minimum), nếu có đủ đà, quả bóng có thể "vượt" qua nó để tiếp tục đi xuống dốc.

Momentum based Gradient Descent Update Rule

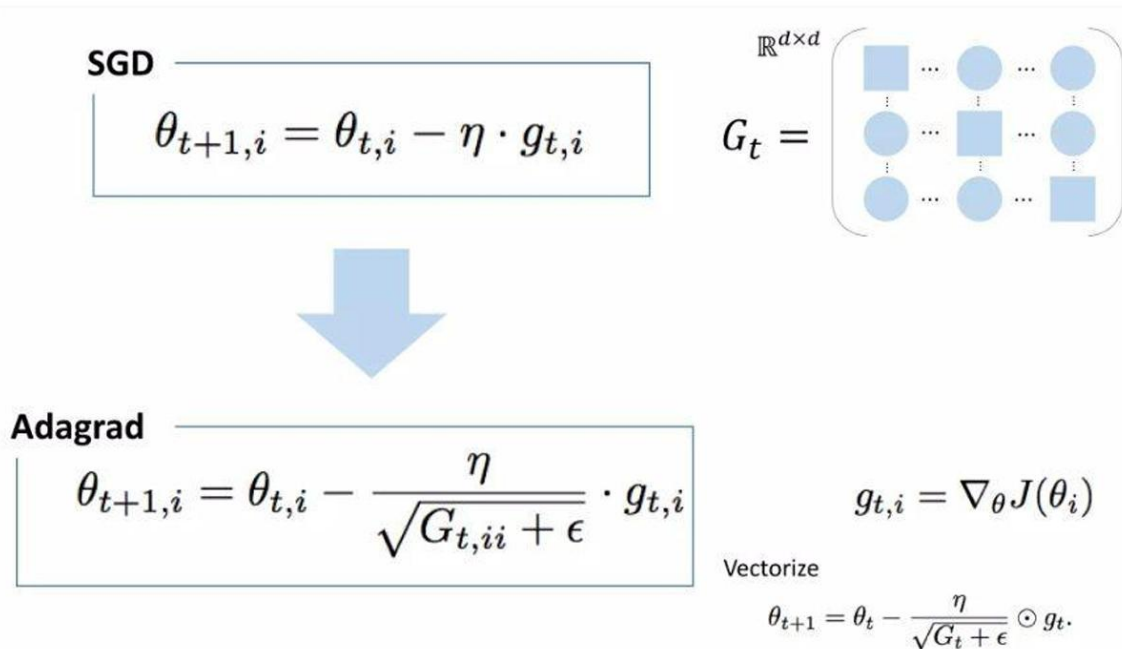
$$v_t = \gamma * v_{t-1} + \eta \nabla w_t$$

$$w_{t+1} = w_t - v_t$$

- Thay vì chỉ dùng gradient hiện tại, Momentum giới thiệu một biến mới là **vận tốc (velocity)** v .
 - v_t là vận tốc tại thời điểm t . Ban đầu, vận tốc v_0 được khởi tạo, thường là bằng **0** (hoặc một giá trị nhỏ tùy theo bài toán).
 - γ là hệ số ma sát (momentum), giá trị của γ giúp điều chỉnh sự ảnh hưởng của vận tốc cũ. Nếu γ gần 1, vận tốc cũ sẽ có ảnh hưởng lớn hơn.
 - ∇w_t là gradient tại thời điểm t , tức là sự thay đổi của hàm mất mát đối với trọng số w_t .
 - Trọng số tại bước tiếp theo w_{t+1} được cập nhật bằng cách trừ đi vận tốc v_t .
- Nếu không có momentum, thuật toán Gradient Descent có thể gặp vấn đề với việc di chuyển quá nhanh hoặc quá chậm, đặc biệt là trong những khu vực có độ dốc thay đổi mạnh.
- Việc sử dụng momentum giúp duy trì hướng đi ổn định hơn và có thể giúp thuật toán vượt qua các "hố" nhỏ trong không gian tìm kiếm.
- Nói cách khác, Momentum giúp thuật toán Gradient Descent "**tiến xa hơn**" trong các khu vực có độ dốc lớn và "**giảm tốc**" khi cần thiết trong các khu vực phẳng.

4.4.3.4. Adaptive Gradient Descent (Adagrad)

- là một biến thể của **Stochastic Gradient Descent (SGD)**, giúp tự động điều chỉnh tốc độ học (learning rate) cho từng tham số dựa trên **lịch sử gradient trong quá khứ**.



- **Trong đó:**

- $\theta_{t,i}$: giá trị tham số (parameter) thứ i tại bước lặp thứ t .
- $g_{t,i} = \nabla_{\theta_i} J(\theta)$: **gradient** của hàm mất mát $J(\theta)$ theo tham số θ_i .
- η : tốc độ học (learning rate) cố định cho mọi tham số.
- G_t : **ma trận chéo (diagonal matrix)** kích thước $d \times d$, trong đó:

$$G_{t,ii} = \sum_{\tau=1}^t (g_{\tau,i})^2$$

tức là tổng bình phương của tất cả gradient cho tham số θ_i từ đầu đến thời điểm t .

- Ma trận chéo (diagonal matrix) là một loại ma trận vuông đặc biệt, trong đó tất cả các phần tử ngoài đường chéo chính đều bằng 0. Các phần tử trên đường chéo chính có thể khác 0.
- Giả sử mô hình của bạn có **3 tham số**: $\theta = [\theta_1, \theta_2, \theta_3]$
- Mỗi lần bạn huấn luyện (một bước gradient descent), bạn tính được **gradient** tương ứng: $g_t = [g_{t,1}, g_{t,2}, g_{t,3}]$
- $G_{t,11}$ = tổng bình phương các gradient của θ_1 qua mọi lần cập nhật.
- $G_{t,22}$ = tổng bình phương các gradient của θ_2 ...
- G_t là **ma trận chéo (diagonal matrix)**:

$$G_t = \begin{bmatrix} G_{t,11} & 0 & 0 \\ 0 & G_{t,22} & 0 \\ 0 & 0 & G_{t,33} \end{bmatrix}$$

- Khi $G_{t,ii}$ **lớn** $\rightarrow$ tham số đó đã có **nhều gradient mạnh** trong quá khứ $\rightarrow$ giảm bước nhảy (learning rate nhỏ hơn).
- Khi $G_{t,ii}$ **nhỏ** $\rightarrow$ tham số đó ít thay đổi $\rightarrow$ tăng bước nhảy (learning rate lớn hơn).
- ϵ : hệ số rất nhỏ (thường 10^{-8}) giúp **tránh chia cho 0**.
- **Phù hợp cho dữ liệu thưa** (sparse data) như trong NLP (vì các tham số hiếm khi được cập nhật sẽ có learning rate cao hơn).
- Tổng bình phương gradient ($G_{t,ii}$) **tăng liên tục**, khiến mẫu số $\sqrt{G_{t,ii}}$ ngày càng lớn $\rightarrow$ **learning rate giảm dần về 0** $\rightarrow$ có thể **ngừng học quá sớm**.

4.4.3.5. Root Mean Square Propagation (RMSProp)

- là một thuật toán **tối ưu hóa thích nghi (adaptive optimizer)** được phát triển dựa trên **Adagrad**, nhằm khắc phục nhược điểm khiến Adagrad làm giảm learning rate quá nhanh.
- RMSProp không cộng dồn toàn bộ gradient cũ, mà chỉ **giữ lại một phần “trung bình trượt” (moving average)** của các gradient gần đây.
- **Công thức:**

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{(1-\gamma)g_{t-1}^2 + \gamma g_t^2 + \epsilon}} \cdot g_t$$

γ : Hệ số “giảm trọng số” (decay rate), thường ≈ 0.9 . Quyết định mức độ nhớ gradient cũ.

- RMSProp giống như “phanh tự động” thông minh cho mỗi tham số, chỉ phanh mạnh khi gradient đang dao động lớn.

- Một thuật toán còn phổ biến hơn nữa là **Adam (Adaptive Moment Estimation)**, nó kết hợp cả ý tưởng của **Momentum** và **RMSProp** và thường là lựa chọn mặc định hàng đầu cho hầu hết các bài toán Deep Learning ngày nay.

4.4.4. BackPropagation (Lan truyền ngược)

- **Cost function được sử dụng cho lan truyền ngược:**

$$C = \frac{1}{2n} \sum_x \|y(x) - a^L(x)\|^2$$

n : number of samples

x : input of individual sample

$$x = (x_1, x_2, \dots)$$

y : output corresponding to input x

$$y = (y_1, y_2, \dots)$$

- Đối với một mẫu dữ liệu đầu vào: $C = \frac{1}{2} \|y - a^L\|^2$, với $a^L = \hat{y}$.
- **Cost function này là biến thể của MSE**, thêm hệ số 1/2 để cho đạo hàm gọn hơn khi làm backpropagation:

- **Nếu không có 1/2:** $\frac{d}{dy} (y - \hat{y})^2 = -2(y - \hat{y})$

- **Nếu có 1/2:** $\frac{d}{dy} \frac{1}{2} (y - \hat{y})^2 = -(y - \hat{y})$

- Mục tiêu của lan truyền là đi tìm **W, b** mới để áp dụng cho mô hình sao cho **Cost function nhỏ dần đến khi đạt giá trị nhỏ nhất**.
- Ta sẽ áp dụng ý tưởng của Gradient Descent để cập nhật **W và b**, tức là ta phải đi tìm đạo

hàm của $C(w_{jk}^l)$ và $C(b_j^l)$, hay $\frac{\partial C}{\partial w_{jk}^l}$ và $\frac{\partial C}{\partial b_j^l}$.

- Nền tảng toán học của Backpropagation là **Quy tắc chuỗi (Chain Rule)** trong giải tích.
- Nếu ta có hai hàm **f** và **g** đều **khả vi (differentiable) (có đạo hàm)**, và ta có một hàm hợp: $F(x) = f(g(x))$ thì hàm $F(x)$ cũng khả vi, và đạo hàm của nó được tính theo công thức:

$$F'(x) = f'(g(x)) \cdot g'(x)$$
- Nếu a là một hàm theo b , b lại là hàm theo c , c là hàm theo d , ..., và cuối cùng g là hàm theo x ,

$$a(b(c(d(e(f(g(x)))))))$$

thì đạo hàm của a theo x được tính bằng cách nhân tất cả các đạo hàm riêng lẻ lại với nhau.

$$\frac{da}{dx} = \frac{da}{db} \times \frac{db}{dc} \times \frac{dc}{dd} \times \frac{dd}{de} \times \frac{de}{df} \times \frac{df}{dg} \times \frac{dg}{dx}$$

Hay: $a'(x) = a'(b) \cdot b'(c) \cdot c'(d) \cdot d'(e) \cdot e'(f) \cdot f'(g) \cdot g'(x)$

- **Ví dụ:**

$$\frac{d}{dx} \left[\underbrace{(4x^3 - 5x^2 + 6x - 7)^{10}}_{\text{outside function}} \right] = 10 \underbrace{(4x^3 - 5x^2 + 6x - 7)^9}_{\text{inside function}} \cdot (12x^2 - 10x + 6)$$

$$\frac{dy}{dx} = 10(12x^2 - 10x + 6)(4x^3 - 5x^2 + 6x - 7)^9$$

$$\frac{dy}{dx} = 20(6x^2 - 5x + 3)(4x^3 - 5x^2 + 6x - 7)^9$$

- Ta sẽ dựa vào **quy tắc chuỗi** để xây dựng công thức tính $\frac{\partial C}{\partial w_{jk}^l}$ và $\frac{\partial C}{\partial b_j^l}$.

- Lưu ý:** In backpropagation, all values a_j^l, z_j^l and y are available.

- Ta có:** Cost = $C(\sigma(Z(WX, B)))$, với σ là hàm kích hoạt.

- Nên $\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial a_j^l} \cdot \frac{\partial a_j^l}{\partial z_j^l} \cdot \frac{\partial z_j^l}{\partial w_{jk}^l}$, mà $\frac{\partial C}{\partial z_j^l} = \frac{\partial C}{\partial a_j^l} \cdot \frac{\partial a_j^l}{\partial z_j^l} \Rightarrow \boxed{\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial z_j^l} \cdot \frac{\partial z_j^l}{\partial w_{jk}^l}}$

Mà:

Constant when w_{jk}^l is variable

$$z_j^l = \sum_k w_{jk}^l \cdot a_k^{l-1} + b_j^l = \underbrace{w_{j1}^l \cdot a_1^{l-1} + w_{j2}^l \cdot a_2^{l-1} + \dots}_{\text{Constant when } w_{jk}^l \text{ is variable}} + \underbrace{w_{jk}^l \cdot a_k^{l-1} + \dots + b_j^l}_{\text{Constant when } w_{jk}^l \text{ is variable}}$$

$$\frac{\partial z_j^l}{\partial w_{jk}^l} = a_k^{l-1}$$

$$\boxed{\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial z_j^l} a_k^{l-1}}$$

- Tương tự ta lại có:

Constant when b_j^l is variable

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial b_j^l}$$

$$z_j^l = \sum_k w_{jk}^l \cdot a_k^{l-1} + b_j^l = w_{j1}^l \cdot a_1^{l-1} + w_{j2}^l \cdot a_2^{l-1} + \dots + w_{jk}^l \cdot a_k^{l-1} + \dots + b_j^l$$

$$\frac{\partial z_j^l}{\partial b_j^l} = 1$$

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l}$$

- Sai số của node thứ j trong lớp thứ l:

$$\delta_j^l = \frac{\partial C}{\partial z_j^l}$$

$$\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial z_j^l} a_k^{l-1} \Rightarrow \frac{\partial C}{\partial w_{jk}^l} = \delta_j^l a_k^{l-1} \quad \frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l} \Rightarrow \frac{\partial C}{\partial b_j^l} = \delta_j^l$$

All activations $a_{k,l-1}$ are available during feedforwards, what we need only δ_j^l

For final output layer

$$\delta_j^l = \frac{\partial C}{\partial z_j^l} = \frac{\partial C}{\partial a_j^l} \frac{\partial a_j^l}{\partial z_j^l} = \frac{\partial C}{\partial a_j^l} \sigma'(z_j^l)$$

- Khi đã có được $\frac{\partial C}{\partial w_{jk}^l}$ và $\frac{\partial C}{\partial b_j^l}$, tiến hành cập nhật w và b theo gradient descent:

$$\begin{cases} w_{jk_new}^l = w_{jk_old}^l - \eta \cdot \frac{\partial C}{\partial w_{jk_old}^l} \\ b_{j_new}^l = b_{j_old}^l - \eta \cdot \frac{\partial C}{\partial b_{j_old}^l} \end{cases}, \text{ với } \eta \text{ là learning rate.}$$

- Các công thức **nâng cao** hơn, tham khảo: <https://www.youtube.com/watch?v=YaR6kqSvCJM>

• **Lan truyền ngược (backpropagation) là quá trình:**

- Tính gradient của hàm C đối với từng tham số (trọng số w và bias b).
- Đi từ tầng đầu ra $\rightarrow$ tầng giữa $\rightarrow$ tầng đầu vào (ngược so với chiều lan truyền tiến).
- Mọi gradient đều được tính dựa trên giá trị trọng số và bias hiện tại (chưa cập nhật).
- Sau khi tính xong toàn bộ gradient, mới cập nhật trọng số và bias bằng gradient descent.

• **Ví dụ**

Activation function

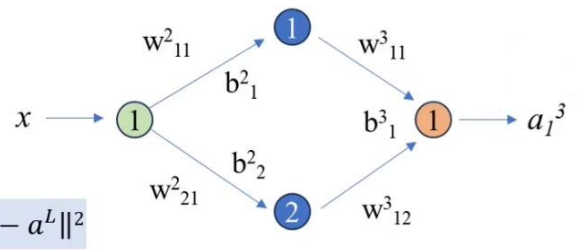
- For hidden layer: sigmoid

$$a(z) = \frac{1}{1 + e^{-z}}$$

	input	output
	H(x)	W(y)
0	160	62
1	165	64
2	170	66

- Cost function

$$C = \frac{1}{2} \|y - a^L\|^2$$



w^2_{11}	w^2_{21}	b^2_1	b^2_2	w^3_{11}	w^3_{12}	b^3_1
0.5	1	0	0	1	0.5	1

(initialize)

$$z^3_1 = w^3_{11} * a^2_1 + w^3_{12} * a^2_2 + b^3_1 = 1 * 1 + 0.5 * 1 + 1 = 2.5$$

$$a^3_1 = z^3_1 = 2.5$$

$$z^2_1 = w^2_{11} * x + b^2_1 = 0.5 * 160 + 0 = 80$$

$$z^2_2 = w^2_{21} * x + b^2_2 = 1 * 160 + 0 = 160$$

$$a^2_1(z^2_1) = \frac{1}{1 + e^{-z^2_1}} = \frac{1}{1 + e^{-80}} = 1$$

$$a^2_2(z^2_2) = \frac{1}{1 + e^{-z^2_2}} = \frac{1}{1 + e^{-160}} = 1$$

- Các giá trị gradient cần tính toán để cập nhật w , b trong lần lan truyền ngược đầu tiên:

$$\frac{\partial C}{\partial w^3_{11}}, \quad \frac{\partial C}{\partial w^3_{12}}, \quad \frac{\partial C}{\partial b^3_1}, \quad \frac{\partial C}{\partial w^2_{11}}, \quad \frac{\partial C}{\partial w^2_{21}}, \quad \frac{\partial C}{\partial b^2_1}, \quad \frac{\partial C}{\partial b^2_2}$$

- Tính toán mẫu cho $\frac{\partial C}{\partial w^3_{11}}$

$$\frac{\partial C}{\partial w^3_{11}} = \frac{\partial C}{\partial a^3_1} \frac{\partial a^3_1}{\partial z^3_1} \frac{\partial z^3_1}{\partial w^3_{11}}$$

$$z^3_1 = w^3_{11} * a^2_1 + w^3_{12} * a^2_2 + b^3_1 \Rightarrow \frac{\partial z^3_1}{\partial w^3_{11}} = a^2_1 = 1$$

$$a^3_1 = z^3_1 \Rightarrow \frac{\partial a^3_1}{\partial z^3_1} = 1$$

$$C = \frac{1}{2} \|y - a^3_1\|^2 \Rightarrow \frac{\partial C}{\partial a^3_1} = (y - a^3_1)(-1) = a^3_1 - y = 2.5 - 62 = -59.5$$

$$w^3_{11_update} = w^3_{11} - \alpha * \frac{\partial C}{\partial w^3_{11}} = 1 + 0.01 * 59.5 = 1.595$$

- Thực hiện tính toán cập nhật cho các giá trị còn lại trong bảng tương tự như trên.

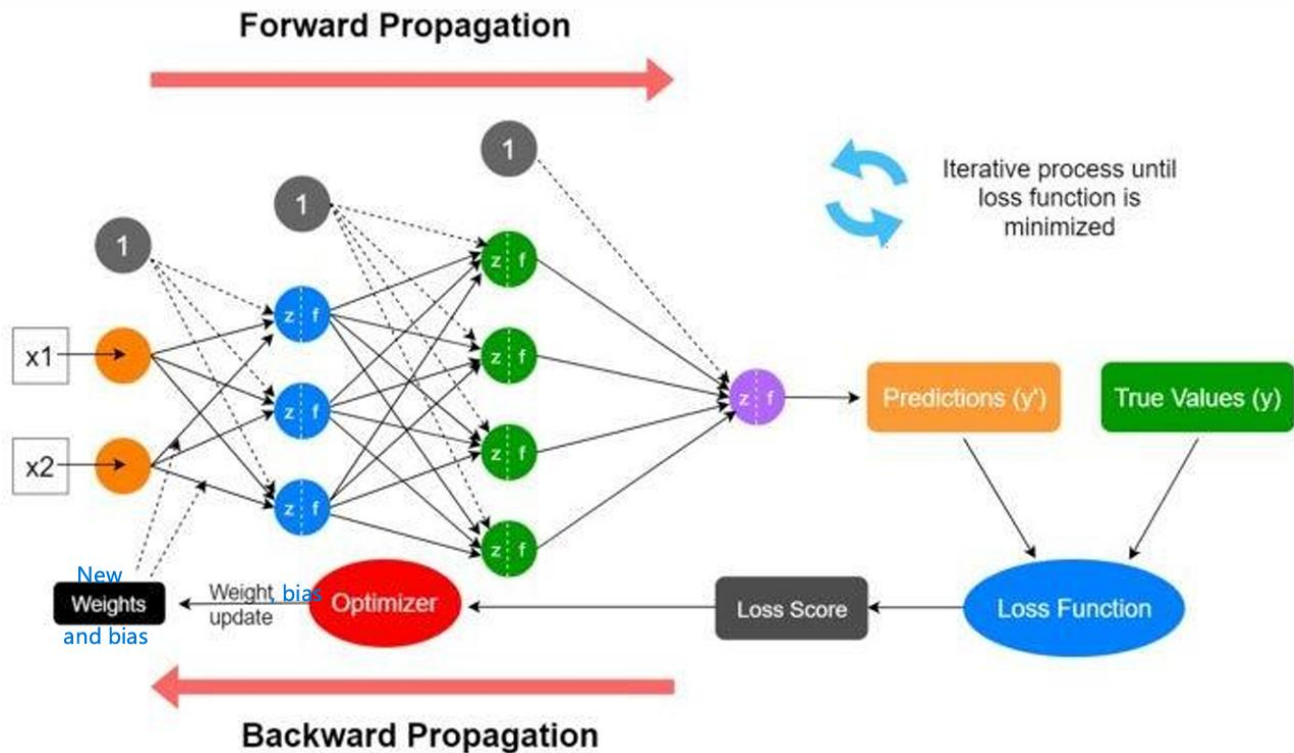
w^2_{11}	w^2_{21}	b^2_1	b^2_2	w^3_{11}	w^3_{12}	b^3_1
0.5	1	0	0	1	0.5	1
●	●	●	●	1.595	●	●

- Sau đó thực hiện tính toán lại các giá trị bên dưới, dựa vào các kết quả đó ta xác định mô hình đã hội tụ hay chưa. Nếu chưa thì lại thực hiện thuật toán lan truyền ngược, cứ thế đến khi mô hình hội tụ.

$$\begin{aligned} z^2_1 &= w^2_{11} * x + b^2_1 & a^2_2(z) &= \frac{1}{1 + e^{-z}} \\ z^2_2 &= w^2_{21} * x + b^2_2 & z^3_1 &= w^3_{11} * a^2_1 + w^3_{12} * a^2_2 + b^3_1 \\ a^2_1(z) &= \frac{1}{1 + e^{-z}} & a^3_1 &= z^3_1 \end{aligned}$$

○ Mô hình hội tụ khi:

- Cost function (C) giảm và gần như chạm ngưỡng nhỏ:
 $C \approx$ nhỏ và không giảm thêm
- Hoặc độ lớn gradient gần bằng 0: $\left| \frac{\partial C}{\partial w} \right| \approx 0, \left| \frac{\partial C}{\partial b} \right| \approx 0$
- Hoặc trọng số cập nhật rất nhỏ: $|w_{new} - w_{old}| \approx 0$
- Hoặc đơn giản hơn: Cost không giảm nữa sau nhiều epoch liên tiếp.

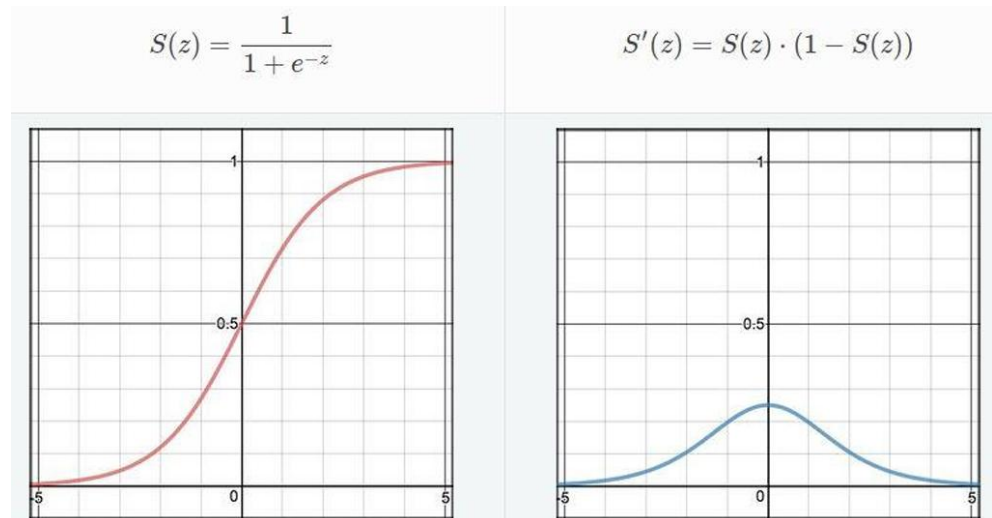


4.5. Các vấn đề và giải pháp khi huấn luyện mạng sâu

4.5.1. Vanishing Gradient (Gradient triệt tiêu)

- **Vanishing Gradient** là một hiện tượng mà các **gradient** trong **thuật toán Gradient Descent** **trở nên quá nhỏ, gần như bằng 0**, khiến cho quá trình tối ưu hóa không thể tiếp tục cải thiện mô hình, và có thể làm cho việc học trở nên chậm chạp hoặc dừng lại hoàn toàn.
- Khi gradient được lan truyền ngược, giá trị gradient tại lớp 1 có thể rất nhỏ, khiến cho các cập nhật trọng số ở lớp đó gần như **không thay đổi**.
- Hay khi gradient bị **nhỏ dần** qua mỗi lớp, mô hình không thể cập nhật các trọng số (weights) ở các lớp sâu, đặc biệt là gần đầu vào → **lớp gần đầu vào** bị ảnh hưởng nghiêm trọng, vì các gradient ở đó trở nên rất nhỏ và không đủ sức mạnh để cập nhật trọng số.
→ Kết quả là việc học trở nên **rất chậm** hoặc **không học được gì** ở một lớp/các lớp sâu.
→ Mô hình sẽ **không học được tốt** và không thể cải thiện hiệu suất trong quá trình huấn luyện.
- **Nguyên nhân của Vanishing Gradient**
 - Trong **mạng nơ-ron**, gradient được tính bằng cách **lan truyền ngược (backpropagation)** từ lớp đầu ra về lớp đầu vào. Khi các gradient này được tính qua nhiều lớp, chúng có thể bị **nhỏ dần** do **đạo hàm của các hàm kích hoạt** (activation functions), đặc biệt là trong các hàm như **sigmoid** và **tanh**.

- **Hàm sigmoid**



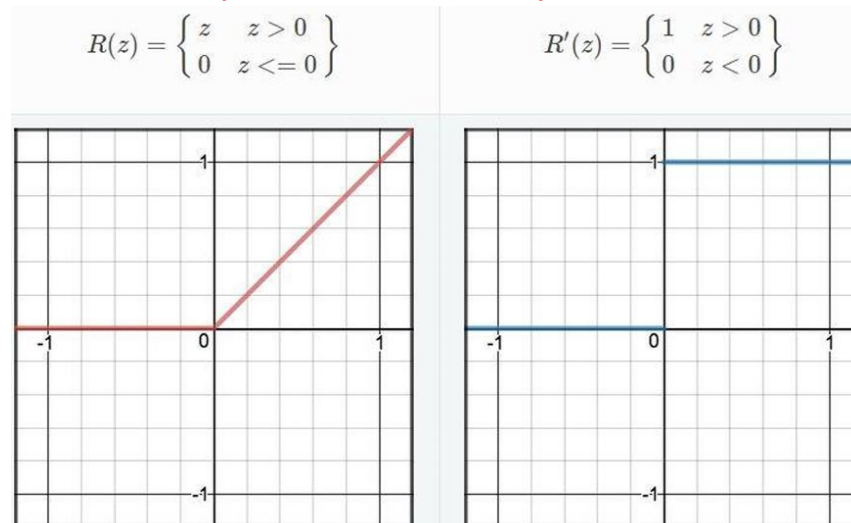
- Trong khoảng giá trị của hàm sigmoid (từ 0 đến 1), đạo hàm có giá trị rất nhỏ.
- **giá trị lớn nhất của đạo hàm này chỉ là 0.25** (tại $z=0$). Khi $|z|$ lớn, đạo hàm tiến về 0, làm cho gradient truyền ngược qua các lớp tiếp theo bị giảm rất nhanh.
- **Hàm tanh** (hyperbolic tangent) cũng gặp phải vấn đề tương tự, với đạo hàm nhỏ khi giá trị vào rất lớn hoặc rất nhỏ.

4.5.2. Exploding gradients (Gradient bùng nổ)

- Ngược lại với Vanishing Gradients.
- **Dấu hiệu:**
 - Các tham số của mô hình tăng rất nhanh (theo cấp số nhân).
 - Model's parameter có thể thành NaN trong quá trình training.
 - Giá trị Loss trở thành NaN (Not a Number) trong quá trình huấn luyện.
 - Loss rất lớn dù đã huấn luyện lâu.
- **Nguyên nhân:**
 - **Improper initial weights (Khởi tạo trọng số không phù hợp):** Nếu các **trọng số ban đầu được khởi tạo với giá trị quá lớn**, khi nhân chúng lại với nhau qua nhiều lớp, kết quả sẽ bùng nổ => **large weight update**.
 - **Poor initial learning rate (Tốc độ học ban đầu tệ):** **Learning rate quá lớn** cũng gây ra cập nhật trọng số quá mạnh.
 - **Poor loss function:** Một số hàm mất mát có thể tạo ra **gradient rất lớn => large loss value => large weight update**.
- Khi gradient quá lớn, việc cập nhật trọng số sẽ giống như một "cú nhảy" khổng lồ trên bề mặt hàm mất mát, khiến **mô hình không bao giờ hội tụ**.

4.5.2. Cách khắc phục Vanishing/Exploding Gradient

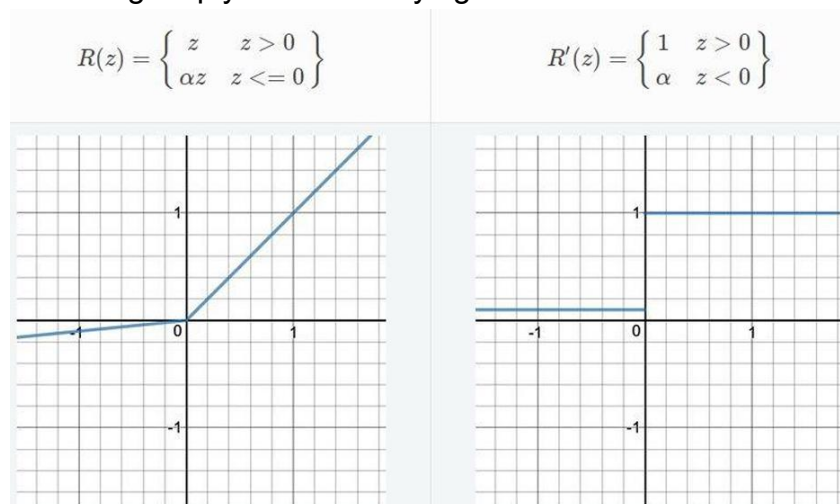
a/ Sử dụng hàm kích hoạt khác - ReLU (Rectified Linear Unit)



- Một lựa chọn phổ biến vì đạo hàm của ReLU luôn là 1 khi giá trị đầu vào dương, giúp tránh được vấn đề gradient bị nhỏ đi quá nhanh.
 - Nếu $z > 0$, đạo hàm bằng 1.
 - Nếu $z < 0$, đạo hàm bằng 0.
- ReLU giúp gradient không bị giảm quá nhỏ, đặc biệt khi các giá trị đầu vào lớn hơn 0.
- Khi neuron được kích hoạt ($z > 0$), gradient truyền qua nó sẽ được nhân với 1, tức là **được giữ nguyên, không bị suy giảm**. Điều này cho phép gradient lan truyền ngược qua nhiều lớp mà không bị triệt tiêu. Đây là một trong những lý do chính khiến ReLU trở thành hàm kích hoạt mặc định trong hầu hết các mạng neural hiện đại.
- **Nhược điểm (Dying ReLU):** Nếu một neuron có đầu vào luôn âm, gradient của nó sẽ luôn là 0, và trọng số của nó sẽ không bao giờ được cập nhật. Neuron đó bị "chết".

• Leaky ReLU

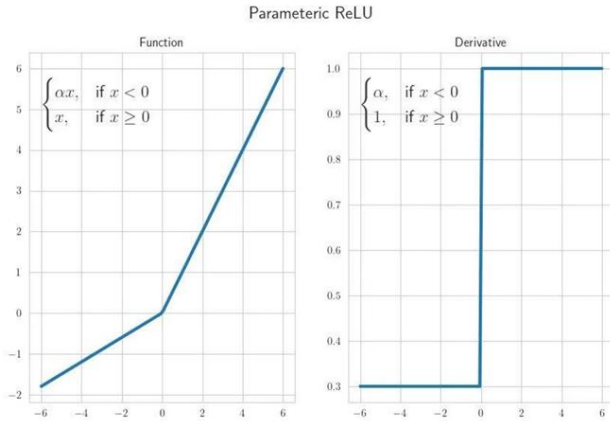
- Một cải tiến của ReLU để giải quyết vấn đề "Dying ReLU".



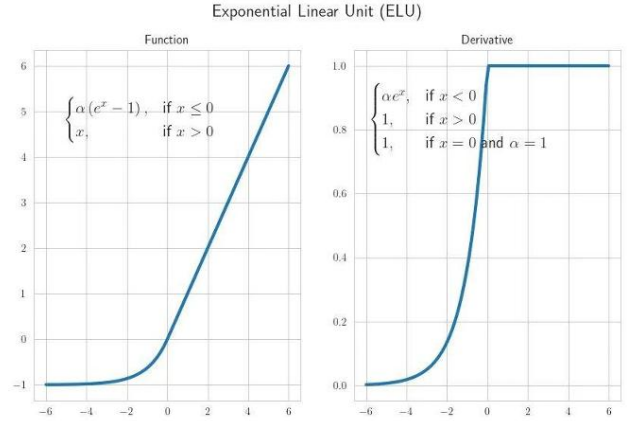
- Nếu $z > 0$, đạo hàm bằng 1.
- Nếu $z < 0$, đạo hàm bằng α (một số dương nhỏ).
- Ngay cả khi neuron không được kích hoạt ($z < 0$), nó vẫn có một gradient nhỏ khác 0. Điều này đảm bảo rằng trọng số vẫn có thể được cập nhật, giúp neuron không bị "chết".

• Các biến thể khác của ReLU

Parametric ReLU

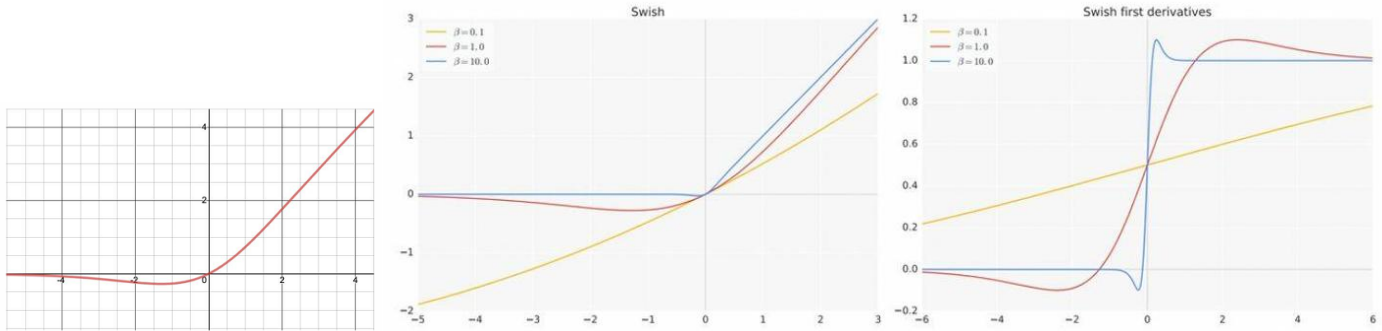


Exponential Linear Unit (ELU)



- **Parametric ReLU (PReLU)**: α không phải là hằng số mà là một tham số được học
- **Exponential Linear Unit (ELU)**, sử dụng hàm mũ cho phần âm để đường cong mượt hơn.

• Hàm kích hoạt SwishL:



- **$f(x) = x * \text{sigmoid}(\beta x)$** .
- Đây là một hàm kích hoạt được Google tìm ra thông qua tìm kiếm tự động, cho thấy hiệu quả tốt hơn ReLU trên một số kiến trúc mạng sâu.

b/ Sử dụng Batch Normalization:

- Giúp điều chỉnh phân phối của đầu vào cho mỗi lớp, giảm thiểu sự thay đổi cực đoan trong các giá trị, giúp cải thiện quá trình lan truyền ngược và giúp giảm thiểu vanishing/ exploding gradient.
- **Ổn định gradient**, Giúp training **nhANH hơn**
- **Ý tưởng**:
 - Ở mỗi layer, thay vì đưa thẳng giá trị $z = w \cdot x + b$ vào activation, ta **chuẩn hóa** chúng về phân phối trung bình 0, phương sai 1: $\hat{z} = \frac{z - \mu}{\sigma}$
 - Sau đó học thêm 2 tham số: $y = \gamma \hat{z} + \beta$, với γ, β là **trainable parameters** (được học như weights).
- **Vì sao giúp chống vanishing/exploding?**
 - Khi chuẩn hóa trước khi đi qua activation, **đầu vào không quá lớn hoặc quá nhỏ** → Gradient ổn định hơn.

c/ Regularization:

- Làm mô hình **đơn giản hơn**, không học quá sát dữ liệu
- Giải quyết overfitting
- Không trực tiếp giải quyết vanishing/exploding gradients, nhưng gián tiếp **giúp weights không trở nên quá lớn** → ổn định hơn.
- **Ý tưởng**: Thêm một "hình phạt" vào **Loss Function**, để mô hình không cho trọng số (weights) quá lớn.
- **Ví dụ: L2 Regularization (Ridge)**: $C' = C + \lambda \sum w^2$
 - C : Loss gốc
 - λ : Hệ số điều chỉnh (hyperparameter)
→ Khi weights lớn → bị phạt → mô hình "giữ weights nhỏ".

d/ Khởi tạo trọng số tốt hơn:

- Khởi tạo trọng số phù hợp có thể giúp giảm bớt vấn đề vanishing/exploding gradient.
- Nếu **trọng số quá nhỏ** → tín hiệu truyền qua nhiều lớp bị *teo dần* → **Vanishing gradient**.
- Nếu **trọng số quá lớn** → tín hiệu bị *phóng đại* qua nhiều lớp → **Exploding gradient**.
- **Các cách khởi tạo**:
 - **✗ Zero/Constant initialization**
 - Nếu tất cả các trọng số của các neuron trong cùng một lớp được khởi tạo giống hệt nhau (ví dụ, bằng 0), thì **khi lan truyền tiến, tất cả các neuron đó sẽ cho ra cùng một kết quả**.
 - Khi lan truyền ngược, chúng cũng sẽ nhận được cùng một upstream gradient và có cùng local gradient. Do đó, chúng sẽ **được cập nhật một lượng y hệt nhau**.
 - **Hậu quả**: Tất cả các neuron trong lớp sẽ "học" cùng một thứ và không bao giờ khác biệt được. Điều này được gọi là **"fail to break symmetry" (thất bại trong việc phá vỡ tính đối xứng)**. Mạng neural mất đi sức mạnh của nó, vì nó không khác gì chỉ có một neuron duy nhất trong mỗi lớp.
 - **Random initialization (Khởi tạo ngẫu nhiên)**
 - Giải quyết được vấn đề break the symmetry. Mỗi neuron sẽ bắt đầu ở một điểm khác nhau và học các đặc trưng khác nhau.
 - Khởi tạo các trọng số với các giá trị ngẫu nhiên nhỏ (lấy từ phân phối Normal hoặc Uniform).
 - Tuy nhiên nếu chúng ta chọn phân phối ngẫu nhiên không phù hợp (ví dụ, các giá trị quá lớn hoặc quá nhỏ), nó vẫn có thể dẫn đến **Vanishing/Exploding Gradients**.
 - **Xavier/Glorot initialization**
 - Dùng khi activation function ở các lớp fully connected, conv là **sigmoid hoặc tanh**.
 - Lớp **Fully Connected (FC)** hay **Dense layer** là lớp mà **mỗi neuron đầu ra được kết nối với tất cả các neuron đầu vào**.
 - Lớp Convolution (Conv Layer)(**tích chập**) dùng trong mạng CNN để **trích xuất đặc trưng không gian** (spatial features) từ ảnh, chuỗi, v.v.
 - Có 2 dạng:

Loại khởi tạo	Dạng	Ký hiệu
Xavier Uniform	$W \sim U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, +\sqrt{\frac{6}{n_{in} + n_{out}}}\right)$	❖ $U(a, b)$ — <i>Uniform Distribution (phân phối đều)</i> ❖ Dấu $\sim$ nghĩa là "được lấy theo phân phối" ❖ $W \sim U(a, b)$ ○ W được chọn ngẫu nhiên ○ Giá trị nằm trong [a, b] ○ Mọi giá trị trong khoảng này có xác suất như nhau ❖ Ví dụ: $W \sim U(-0.1, 0.1) \rightarrow$ W có thể là bất kỳ số nào từ -0.1 đến 0.1, khả năng như nhau.
Xavier Normal	$W \sim N\left(0, \frac{2}{n_{in} + n_{out}}\right)$	❖ $N(\mu, \sigma^2)$ — <i>Normal Distribution (phân phối chuẩn / Gaussian)</i> ❖ Trong đó: • μ (mean) = trung bình • σ^2 (variance) = độ phân tán quanh mean ❖ Ví dụ: $W \sim N(0, 0.01) \rightarrow$ W sẽ là các số <i>nhỏ, tập trung quanh 0</i>, đa phần gần 0, một vài giá trị xa hơn.

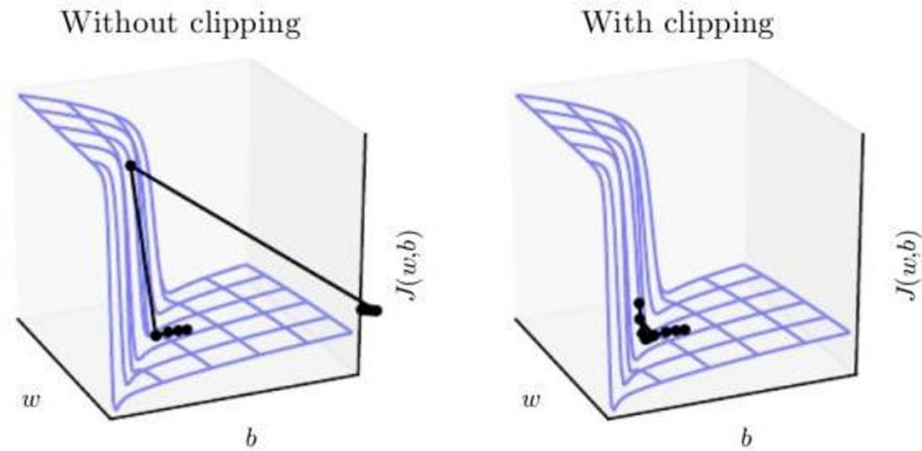
○ **He/Kaiming initialization**

- Một cải tiến của Xavier, được thiết kế riêng cho **ReLU**.
- **Vấn đề với Xavier và ReLU:** Hàm ReLU "cắt" một nửa phân phối (phần âm), làm giảm phương sai đi một nửa. Nếu dùng Xavier, phương sai sẽ giảm dần qua các lớp, dẫn đến vanishing gradient.
- **Giải pháp của He/Kaiming:** Để bù lại việc phương sai bị giảm một nửa bởi ReLU, phương pháp He chỉ cần nhân đôi phương sai của phân phối khởi tạo.
- **Áp dụng cho:** Các lớp sử dụng hàm kích hoạt **ReLU** và các biến thể của nó.
- Có 2 dạng:

He Uniform	$W \sim U\left(-\sqrt{\frac{6}{n_{in}}}, +\sqrt{\frac{6}{n_{in}}}\right)$
He Normal	$W \sim N\left(0, \frac{2}{n_{in}}\right)$

e/ Gradient Clipping:

- Trong một số trường hợp, **gradient clipping** có thể giúp giới hạn giá trị gradient khi chúng quá lớn hoặc quá nhỏ, giúp huấn luyện ổn định hơn.
- Ý tưởng: Nếu gradient quá lớn $\rightarrow$ **cắt / thu nhỏ** nó lại.



- **Without clipping (bên trái):** Khi gradient rất lớn (do gặp phải một "vách đá" dốc đứng trên bề mặt loss), bước cập nhật sẽ rất lớn, "ném" các tham số đi rất xa, có thể đến một vùng còn tệ hơn.
- **With clipping (bên phải):** Thuật toán sẽ kiểm tra độ lớn của vector gradient. **Nếu nó vượt quá một ngưỡng đã định trước, vector gradient sẽ được "co lại" để có độ lớn bằng đúng ngưỡng đó (nhưng vẫn giữ nguyên hướng).** Điều này tạo ra một bước cập nhật nhỏ hơn, có kiểm soát, giúp mô hình đi xuống dốc một cách ổn định hơn.

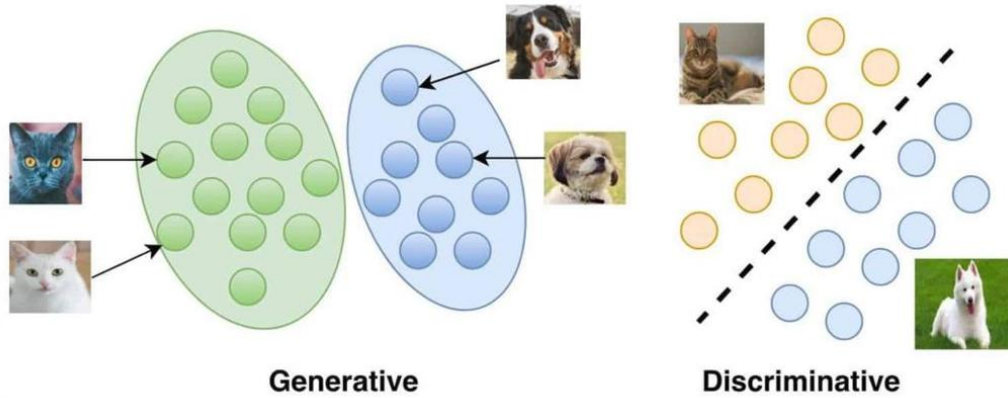
f/ Sử dụng các mô hình khác:

- Các kiến trúc **mạng nơ-ron hồi tiếp (RNN)** hoặc **LSTM (Long Short-Term Memory)** có thể giải quyết vấn đề này trong các bài toán chuỗi thời gian.

4.6. Các chủ đề nâng cao

4.6.1. Hai trường phái chính trong Machine Learning

- Mô hình Sinh (Generative) và Mô hình Phân biệt (Discriminative).



	Discriminative model	Generative model
Goal	Directly estimate $P(y x)$	Estimate $P(x y)$ to then deduce $P(y x)$
What's learned	Decision boundary	Probability distributions of the data
Illustration		
Examples	Regressions, SVMs	GDA, Naive Bayes

Discriminative Model (Mô hình Phân biệt):

- Mục tiêu:** Học cách **phân chia** các nhóm dữ liệu. Nó chỉ tập trung vào việc tìm ra **ranh giới (decision boundary)** giữa các lớp.
- Trả lời câu hỏi: Dữ liệu này thuộc lớp nào?**
- Nghĩa là, khi đưa vào **x (dữ liệu)** → mô hình dự đoán **y (nhãn)**
- Không cố gắng hiểu dữ liệu sinh ra như thế nào.
- "Cho một bức ảnh, nó thuộc lớp Mèo hay lớp Chó?"
- Cách học:** Nó học xác suất có điều kiện $P(y|x)$ - "Xác suất của nhãn y khi biết đầu vào x là gì?".
- Ví dụ:** Hầu hết các mô hình bạn đã học (Logistic Regression, SVM, CNN cho bài toán phân loại) đều là mô hình phân biệt.

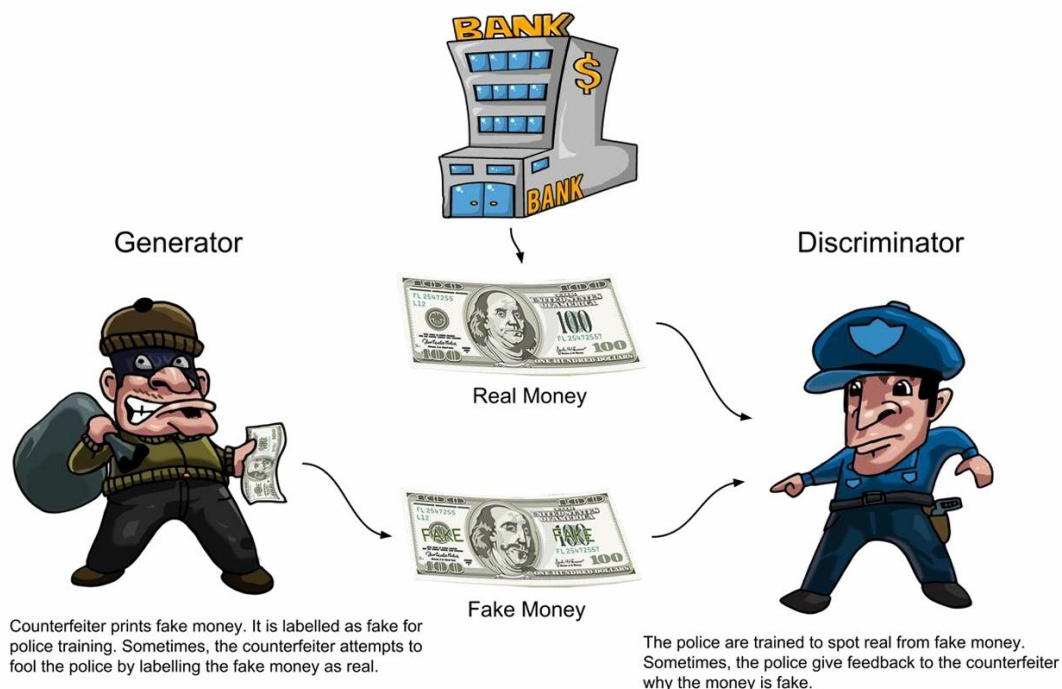
Generative Model (Mô hình Sinh):

- Mục tiêu:** Học cách **hiểu và tái tạo** phân phối dữ liệu (**Học cách dữ liệu được tạo ra**). Nó không chỉ học ranh giới, mà học **"bản chất"** của từng nhóm dữ liệu **trông như thế nào**.
- Tức là học **đặc trưng** của từng lớp, rồi mới phân loại sau.
- Cho phép **tái tạo / sinh ra** dữ liệu giống dữ liệu thật.
- Trả lời câu hỏi: Dữ liệu của từng lớp trông như thế nào?**
- "Một bức ảnh Mèo trông như thế nào?" (mèo lông xanh, mèo mắt nhỏ...)
- Cách học:** Nó học xác suất đồng thời $P(x, y)$ - "Xác suất để một cặp dữ liệu x và nhãn y cùng xuất hiện là bao nhiêu?". Khi đã học được điều này, nó có thể **sinh ra** một x mới (một bức ảnh mèo mới).
- Ví dụ:** Naive Bayes, GANs.

Discriminative Model	Generative Model
❖ Mô hình chỉ quan tâm đường phân cách giữa lớp $y = 0$ và $y = 1$. ❖ Nó chỉ học cách phân loại, chứ không học cách tạo ra dữ liệu. Mô hình phân biệt giống như một học sinh chỉ cần học cách khoanh trắc nghiệm. Em đó chỉ cần biết ranh giới giữa "Đúng" và "Sai".	❖ Mô hình học phân bố của dữ liệu lớp $y=0$ và lớp $y=1$ riêng biệt. ❖ Sau đó, khi đưa một mẫu x vào $\rightarrow$ xem mẫu đó hợp với phân bố lớp nào hơn. Mô hình sinh giống như một học sinh phải học để viết một bài luận hoàn chỉnh. Em đó phải hiểu sâu sắc về chủ đề để có thể tự mình tạo ra nội dung mới.
Mô hình sinh khó hơn rất nhiều. Việc học được toàn bộ phân phối dữ liệu để có thể tạo ra các mẫu mới phức tạp hơn nhiều so với việc chỉ kẻ một đường ranh giới.	

4.6.2. Generative Adversarial Networks (GANs) (Mạng Sinh Đối Ngịch)

- Thay vì **phân loại** dữ liệu có sẵn, GANs học cách **tạo ra (sinh ra)** dữ liệu mới chưa từng tồn tại.
- GAN bao gồm **hai mạng neural** cạnh tranh với nhau:



❖ **The Generator (G) - Kẻ làm giả (Tên trộm):**

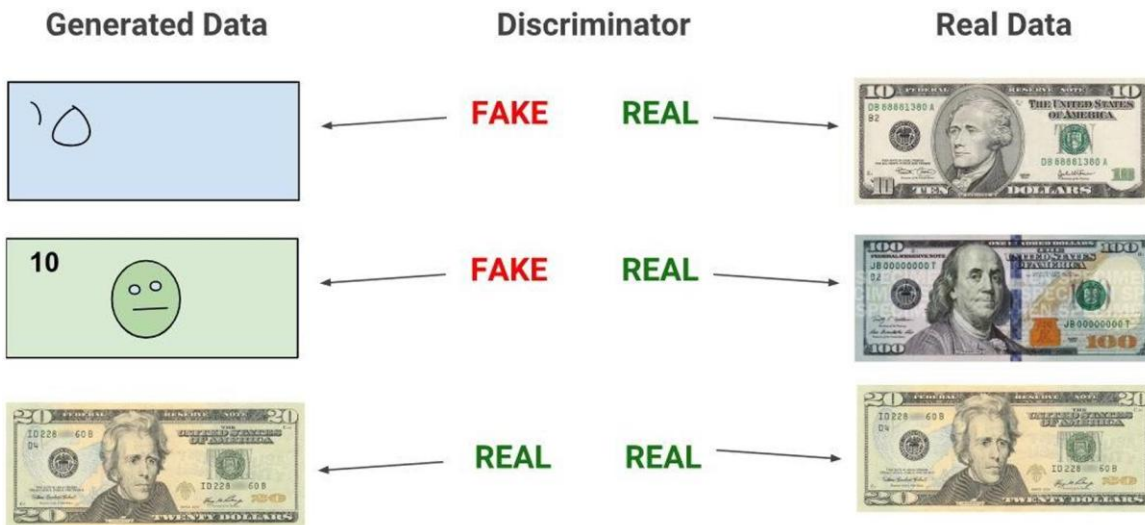
- **Đầu vào:** Một vector nhiễu ngẫu nhiên (random noise).
- **Nhiệm vụ:** Học cách biến nhiễu ngẫu nhiên này thành một sản phẩm giả (ảnh giả, tiền giả) trông y như thật (gán nhãn thật). Mục tiêu của nó là **đánh lừa** được Cảnh sát.
- Cố gắng tạo ra dữ liệu giả trông như thật.

❖ **The Discriminator (D) - Người phân biệt (Cảnh sát):**

- **Đầu vào:** Một sản phẩm (có thể là thật hoặc giả).
- **Nhiệm vụ:** Được huấn luyện với cả dữ liệu thật (tiền thật) và dữ liệu giả từ Generator. Nó phải học cách phân biệt đâu là thật, đâu là giả. Mục tiêu của nó là **không bị lừa**.
- Cố gắng phân biệt giữa dữ liệu thật và dữ liệu giả do Generator tạo ra.

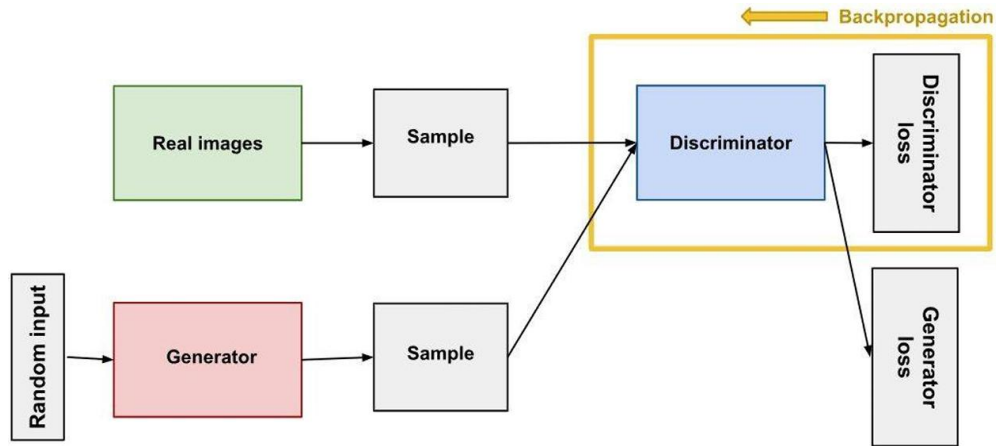
• **Quá trình "Đối nghịch" (Adversarial):**

- Generator cố gắng tạo ra sản phẩm ngày càng tinh vi để lừa Discriminator.
- Discriminator ngày càng tinh mắt hơn để phát hiện ra các sản phẩm giả.
- Chúng là một cặp "kỳ phùng địch thủ". Sự cạnh tranh này đẩy cả hai cùng tiến bộ. Cuối cùng, **khi đạt đến trạng thái cân bằng, Generator sẽ có khả năng tạo ra những sản phẩm giả mà ngay cả Discriminator cũng không thể phân biệt được** (xác suất đoán đúng là 50/50).



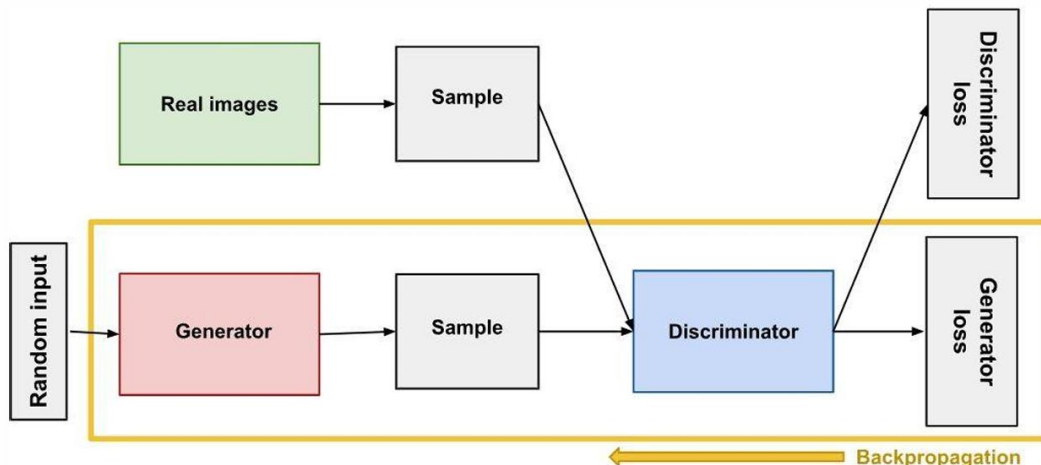
GAN's training: **Lặp lại 2 bước bên dưới:**

- **Discriminator's training (Huấn luyện Discriminator):**



- **Đóng băng (freeze)** Generator (được giữ cố định). Nó **chỉ tạo ra các ảnh giả mà không cập nhật trọng số**.
- Lấy một lô ảnh **thật** và một lô ảnh **giả**.
- Huấn luyện Discriminator: nói với nó rằng lô đầu tiên là "Thật" (nhãn 1) và lô thứ hai là "Giả" (nhãn 0).
- **Discriminator tính loss và cập nhật trọng số của nó thông qua Backpropagation** để trở nên giỏi hơn trong việc phân biệt.
- Discriminator được huấn luyện trong 1 hoặc vài epochs.
- **Discriminator Loss Function:** The discriminator tries to minimize this loss:
$$J_D = -\frac{1}{m} \sum_{i=1}^m \log D(x_i) - \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z_i)))$$
 - J_D measures how well the discriminator classifies real and fake samples.
 - x_i is a **real** data sample.
 - $G(z_i)$ is a **fake** sample from the generator.
 - $D(x_i)$ is the **discriminator's probability that x_i is real**.
 - $D(G(z_i))$ is the **discriminator's probability that the fake sample is real**.
- The discriminator wants to correctly classify real data as real (maximize $\log D(x_i)$) and fake data as fake (maximize $\log(1 - D(G(z_i)))$)

- **Generator's training (Huấn luyện Generator):**



- **Đóng băng** Discriminator (được giữ cố định). Nó chỉ đóng vai trò **thẩm định mà không cập nhật**.
- Generator tạo ra một lô ảnh **giả**.
- Đưa các ảnh giả này vào Discriminator, nhưng lần này chúng ta **đánh lừa** nó bằng cách **gán nhãn là "Thật" (nhãn 1)**.
- Discriminator sẽ đưa ra dự đoán (ví dụ, nó nghĩ đây là giả, xác suất thật là 0.1).
- **Loss được tính dựa trên sự khác biệt giữa mục tiêu (1) và dự đoán (0.1)**.
- Gradient của loss này được lan truyền ngược **CHỈ qua Generator**. Generator sẽ cập nhật trọng số của nó để lần sau, ảnh giả nó tạo ra sẽ khiến Discriminator đưa ra xác suất thật cao hơn (ví dụ 0.2, 0.3,...).
- Generator được huấn luyện trong 1 hoặc vài epochs.
- **Generator Loss Function:** The generator tries to minimize this loss:

$$J_G = -\frac{1}{m} \sum_{i=1}^m \log D(G(z_i))$$

where

- J_G measure how well the generator is fooling the discriminator.
- $G(z_i)$ is the **generated sample from random noise z_i**
- $D(G(z_i))$ is **the discriminator's estimated probability** that the generated sample is real.
- The generator aims to maximize $D(G(z_i))$ meaning it wants the discriminator to classify its fake data as real (probability close to 1).

MinMax Loss

- GANs are trained using a MinMax Loss between the generator and discriminator:

$$\min_G \max_D (G, D) = [\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(g(z)))]]$$

where,

- G is generator network and D is the discriminator network
- $p_{data}(x)$ = true data distribution
- $p_z(z)$ = distribution of random noise (usually normal or uniform)
- $D(x)$ = discriminator's estimate of real data
- $D(G(z))$ = discriminator's estimate of generated data
- **The generator tries to minimize this loss (to fool the discriminator) and the discriminator tries to maximize it (to detect fakes accurately).**

Generative Adversarial Network VS Traditional Machine Learning

 **Synthetic Data**

 **Generator vs Discriminator**

 **Adversarial Training**

 **Realistic Output**

 **Predict Outcomes**

 **Supervised Learning**

 **Minimize Error**

 **Regression & Classification**

Applications of GANs

Image Generation

GANs are widely used in creating realistic images (e.g faces, art).



Super Resolution

GANs improve the resolution of images or videos.



Data Augmentation

GANs generate more data for training machine learning models.



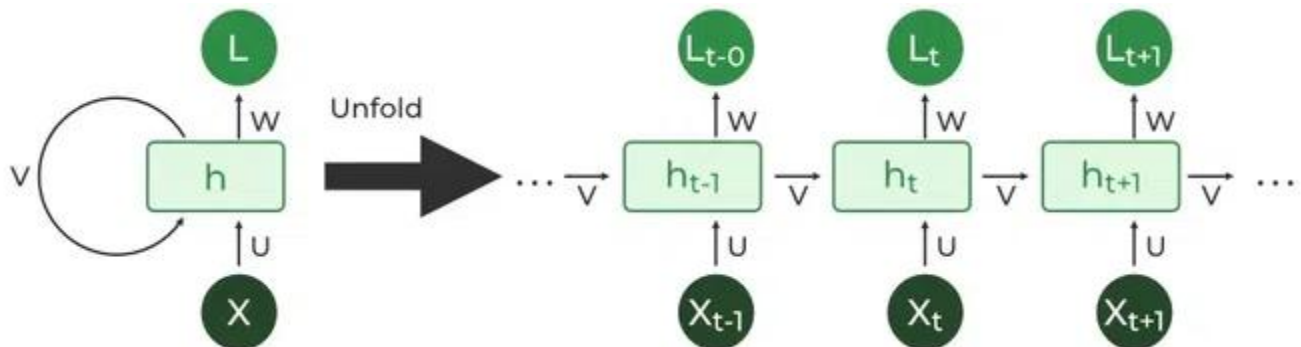
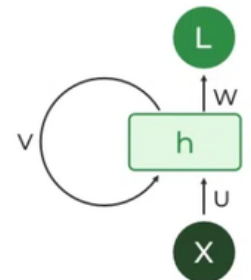
Text-to-Image

GANs convert textual descriptions into corresponding images



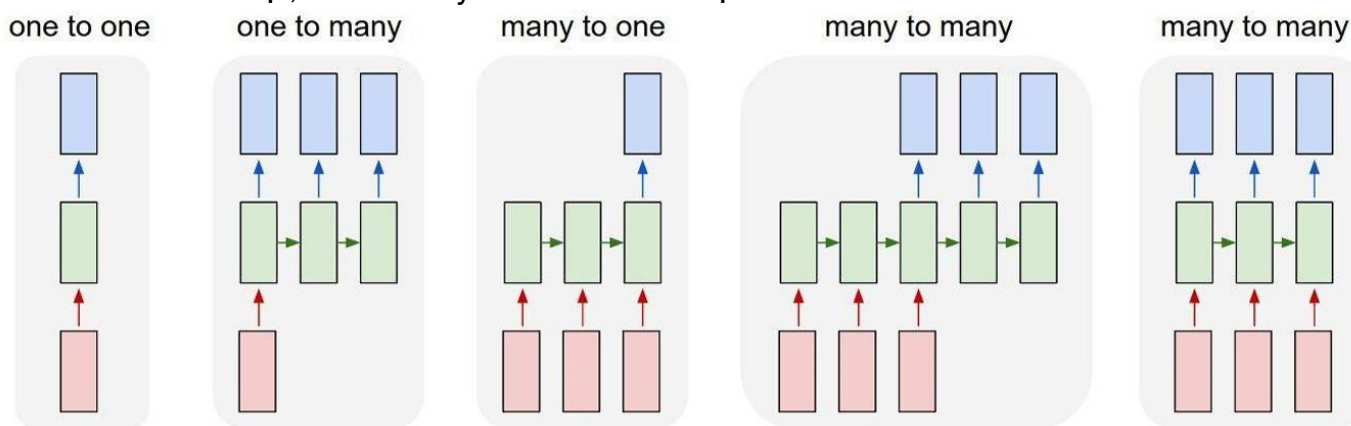
4.6.3. Recurrent Neural Network (RNNs) (Mạng Neural Hồi quy)

- Được thiết kế để xử lý **dữ liệu dạng chuỗi (sequential data)**, nơi thứ tự của các phần tử là quan trọng.
- Các mạng neural thông thường (và cả CNN) **xử lý các đầu vào một cách độc lập**. Chúng **không có "bộ nhớ" về những gì đã xử lý trước đó**. Điều này không phù hợp với các tác vụ như *dịch máy* (nghĩa của một từ phụ thuộc vào các từ đứng trước) hay *dự báo thời tiết* (nhiệt độ ngày mai phụ thuộc vào hôm nay).
- Ex:** Imagine **reading a sentence and you try to predict the next word**, you *don't rely only on the current word but also remember the words that came before*. RNNs work similarly by "remembering" past information and passing the output from one step as input to the next i.e it considers all the earlier words to choose the most likely next word. This memory of previous steps *helps the network understand context and make better predictions*.
- Recurrent Neurons:** The fundamental processing unit in RNN is a Recurrent Unit. They **hold a hidden state that maintains information about previous inputs in a sequence**. Recurrent units can "remember" information from prior steps by feeding back their hidden state, allowing them to **capture dependencies across time**.
- RNN unfolding or unrolling** is the process of expanding the recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.



Ký hiệu	Ý nghĩa
X	Đây là input đầu vào cho mạng RNN. Khi unfold sẽ thành: $x_{t-0}, x_t, x_{t+1}, \dots$
h = các h_t	Đây là trạng thái ẩn (hidden state) tại thời điểm t . Nó <u>lưu giữ “kỷ ức” ngắn hạn từ các bước trước.</u>
L	Là output tại mỗi thời điểm t . Trong bài toán phân loại, đây có thể là lớp dự đoán.
U, V, W	Đây là các ma trận trọng số (weights) được học trong quá trình huấn luyện:
	- U : Trọng số từ input x_t vào hidden state h_t .
	- W : Trọng số từ hidden state h_t đến output L_t .
	- V : Trọng số kết nối hidden state giữa các bước thời gian (tức từ $h_{t-1} \rightarrow h_t$).
Unfold	Nghĩa là “trải” RNN theo thời gian. Ở thực tế RNN là 1 khối lặp lại , nhưng <i>khi vẽ sẽ nhân bản ra theo từng bước thời gian để dễ phân tích và tính gradient.</i>

- RNN rất linh hoạt, có thể xử lý nhiều kiểu ánh xạ đầu vào-đầu ra.



- **one to one:** Mạng neural thông thường.
 - This is the simplest type of neural network architecture where there is a **single input and a single output**.
 - It is used for straightforward classification tasks such as binary classification where no sequential data is involved.
 - **one to many:**
 - processes a single input to produce multiple outputs over time.
 - Mô tả ảnh (đầu vào là 1 ảnh, đầu ra là 1 chuỗi từ).
 - **many to one:**
 - receives a sequence of inputs and generates a single output.
 - Phân tích cảm xúc (đầu vào là 1 chuỗi từ, đầu ra là 1 nhãn: tích cực/tiêu cực).
 - **many to many (1):**
 - processes a sequence of inputs and generates a sequence of outputs.
 - Dịch máy (đầu vào là 1 chuỗi từ tiếng Việt, đầu ra là 1 chuỗi từ tiếng Anh).
 - Phân loại video theo từng khung hình.

- **Ví dụ:** RNN xử lý chuỗi ký tự "h", "e", "l", "l" để dự đoán ký tự tiếp theo "o". Tại mỗi bước, **input layer là one-hot vector của ký tự hiện tại**, và hidden layer (nơi RNN **ghi nhớ ngữ cảnh** của chuỗi) cập nhật trạng thái của nó dựa trên input và trạng thái cũ.

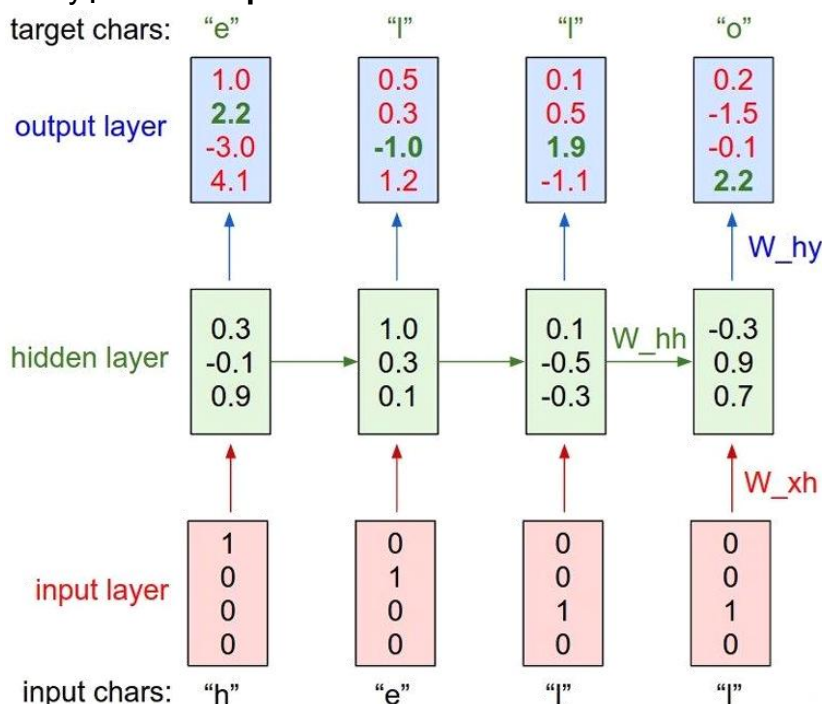
- **W_{xh}** — ma trận trọng số từ Input → Hidden. Ma trận này quyết định input ảnh hưởng thế nào đến trạng thái ẩn.
- **W_{hh}** — ma trận trọng số Hidden → Hidden. giúp RNN *ghi nhớ những gì đã xử lý ở bước trước*. Đây là phần cốt lõi tạo tính "recurrent" (lặp lại).

Công thức:
$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1})$$

- **W_{hy}** — ma trận trọng số Hidden → Output. Ma trận này dùng để chuyển **trạng thái ẩn** → **vector dự đoán đầu ra**.

Output Calculation:
$$y_t = W_{hy} \cdot h_t$$

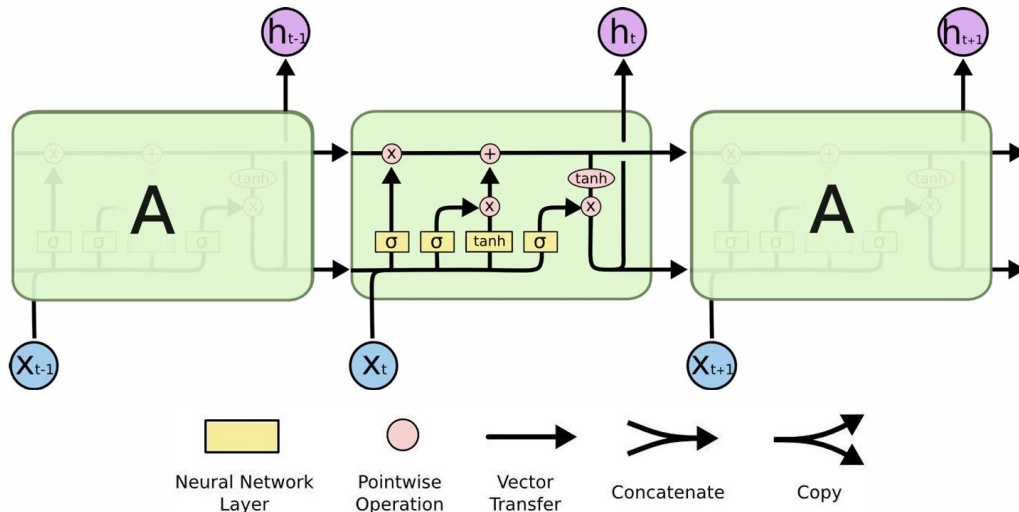
- Các output vector là **logits** (giá trị chưa chuẩn hóa). Sau đó, ta áp dụng **softmax** để chuyển thành xác suất dự đoán ký tự tiếp theo.
- Đây là huấn luyện kiểu **Sequence Prediction / Next Character Prediction**.



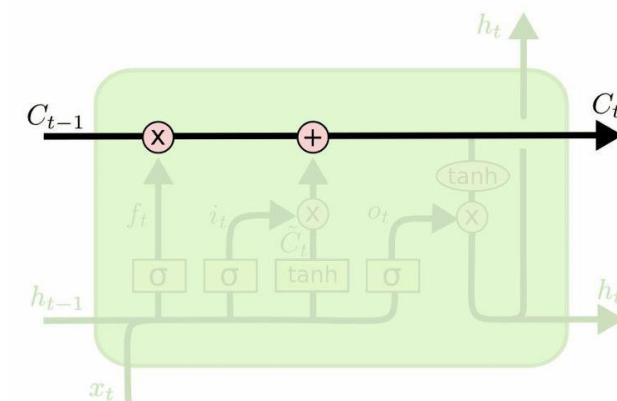
- **Problem of RNN (Vấn đề của RNN):** phụ thuộc xa (long-term dependencies).
 - **Ví dụ 1 (phụ thuộc gần):** "The clouds are in the **sky**". Để dự đoán từ "sky", RNN chỉ cần nhìn vào vài từ ngay trước đó.
 - **Ví dụ 2 (phụ thuộc xa):** "I was born in **Vietnam**... I could speak **Vietnamese** fluently". Để dự đoán từ "Vietnamese", RNN cần phải "nhớ" được thông tin về từ "Vietnam" đã xuất hiện từ rất lâu trước đó.
- **Vấn đề:** RNNs truyền thống (Standard RNN) **gặp khó khăn trong việc lưu giữ thông tin qua nhiều bước thời gian**. Do vấn đề **Vanishing Gradients** (tương tự như mạng sâu), "bộ nhớ" của chúng rất ngắn hạn.

4.6.3.1. Long Short-Term Memory (LSTM)

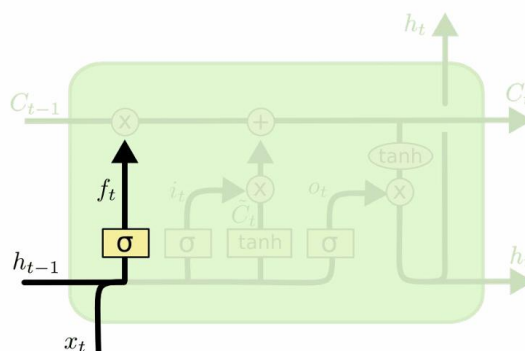
- một loại RNN phức tạp hơn được thiết kế để giải quyết vấn đề phụ thuộc xa.



- Ý tưởng cốt lõi:** LSTM có một "bộ nhớ" dài hạn riêng, gọi là **trạng thái ô (cell state)**, ký hiệu là C_t . Trạng thái này giống như một "băng chuyền", thông tin có thể chạy dọc theo nó mà ít bị thay đổi.

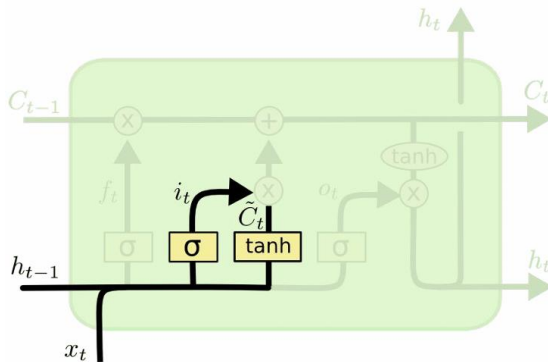


- LSTM sử dụng các cơ chế gọi là "**cổng**" (**gates**) để điều khiển thông tin một cách thông minh. **Cổng** là các mạng neural nhỏ (thường có hàm kích hoạt Sigmoid, cho ra giá trị từ 0 đến 1) để quyết định "bao nhiêu" thông tin được đi qua.
- LSTM có 3 cổng chính:**
 - Forget Gate (Cổng Quên):** Nhìn vào đầu vào hiện tại tại x_t và trạng thái ẩn cũ h_{t-1} , **quyết định thông tin nào trong cell state cũ C_{t-1} cần phải quên đi** (nhân với 0) hoặc giữ lại (nhân với 1).



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

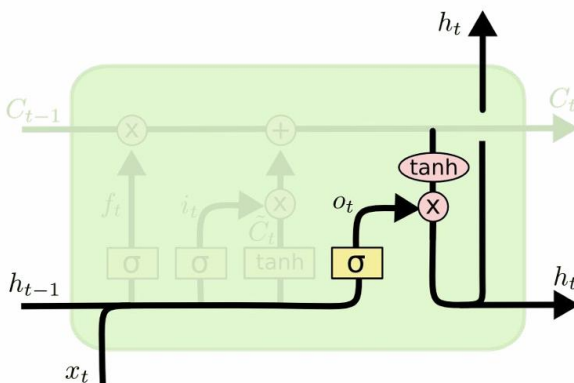
- W_f represents the weight matrix associated with the forget gate.
- $[h_{t-1}, x_t]$ denotes the concatenation of the current input and the previous hidden state.
- b_f is the bias with the forget gate.
- σ is the sigmoid activation function.
- **Input Gate (Cổng Vào):** Quyết định thông tin **mới** nào từ x_t và h_{t-1} là quan trọng và cần được **lưu vào** cell state.



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- **Output Gate (Cổng Ra - Slide 231):** Dựa trên cell state đã được cập nhật C_t , quyết định thông tin nào sẽ được dùng để tạo ra **trạng thái ẩn mới** h_t (đây cũng là đầu ra cho bước hiện tại).



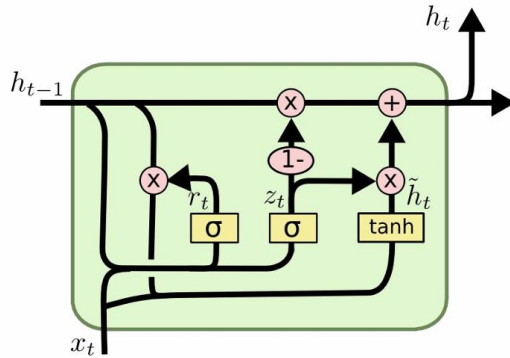
$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

- o_t is the output gate activation.
- C_t is the current cell state.
- σ is the sigmoid activation function.
- **Kết luận:** Nhờ cơ chế cổng này, LSTM có thể học cách **chủ động lưu giữ thông tin quan trọng trong thời gian dài và loại bỏ thông tin không liên quan**, giúp nó giải quyết được vấn đề phụ thuộc xa.

4.6.3.2. Gated Recurrent Unit (GRU)

- Một biến thể khác của RNN có cổng, đơn giản hơn LSTM.



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

- GRU kết hợp Forget Gate và Input Gate thành một cổng duy nhất gọi là **Update Gate**.
- Nó cũng gộp cell state và hidden state làm một.
- So sánh GRU và LSTM:** GRU có ít tham số hơn, tính toán nhanh hơn một chút. Về hiệu suất, không có sự khác biệt rõ rệt, tùy vào bài toán cụ thể. LSTM vẫn phổ biến hơn một chút, nhưng GRU là một lựa chọn thay thế rất tốt.

4.7. Batch size, Iteration & Epoch

Thuật ngữ	Định nghĩa	
Batch size	Số mẫu đưa vào mô hình trong 1 lần cập nhật trọng số	Cập nhật 1 lần
Iteration	Số lần cập nhật trọng số để dùng hết toàn bộ dữ liệu 1 lần	Dùng hết dữ liệu
Epoch	Số lần lặp lại toàn bộ quá trình huấn luyện trên dataset	

Total data : 1,000

Batch size = 100

Iteration = 10

+1 epoch !

- Ví dụ:** Giả sử bạn có:
 Tổng dữ liệu (Total data) = 1000 mẫu,
 Batch size = 100
 - Mỗi batch chứa **100 mẫu**
 - Bạn có 1000 mẫu → chia được thành **10 batch**
 - Vậy có $1000 / 100 = 10$ lần **cập nhật trọng số** để dùng hết dữ liệu → **10 Iterations**
- Khi bạn hoàn thành 10 lần cập nhật đó → bạn vừa xong 1 epoch.**

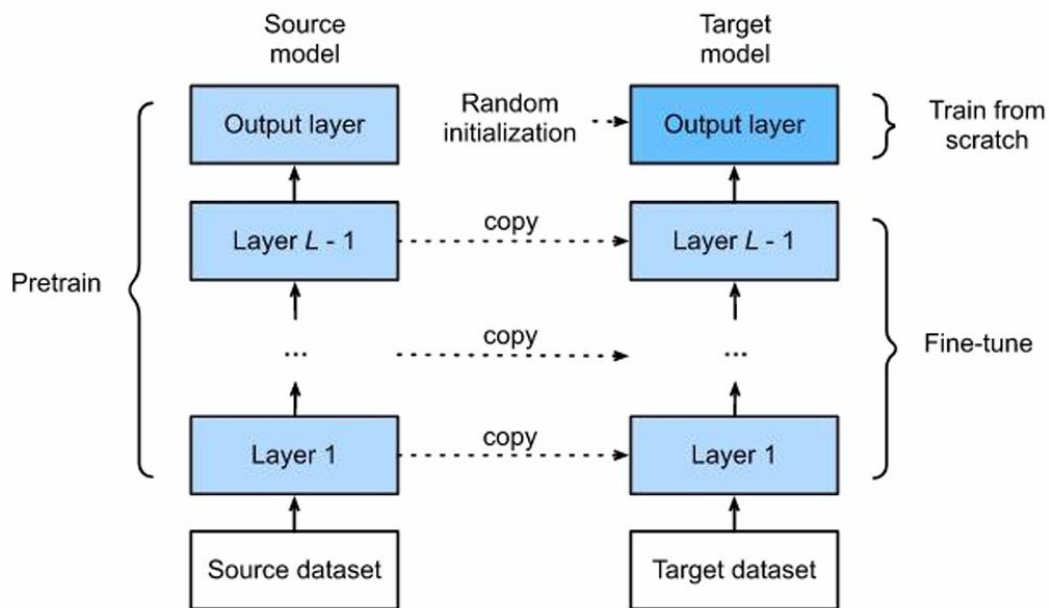
4.8. Fine-tune (tinh chỉnh mô hình)

- **Fine-tune (tinh chỉnh mô hình)** là quá trình **lấy một mô hình đã được huấn luyện trước (pre-trained model)** và **tiếp tục đào tạo nó trên một tập dữ liệu nhỏ hơn, có mục tiêu cụ thể hơn**, thay vì huấn luyện từ đầu.
 - Mô hình **đã có kiến thức chung** → từ dữ liệu lớn.
 - Ta **bổ sung kiến thức chuyên biệt** → bằng dữ liệu riêng của bài toán.→ Điều này giúp **tiết kiệm thời gian, chi phí**, và thường cho **kết quả tốt hơn** so với train từ đầu.
- **Ví dụ**
 - Giả sử bạn có mô hình ngôn ngữ GPT đã học rất nhiều tiếng Anh.
 - Bạn muốn **nó trả lời theo phong cách lịch sự trang trọng** cho email công việc.
 - Bạn không cần train từ đầu → chỉ cần **fine-tune** mô hình bằng một tập dữ liệu nhỏ chứa:

Input email	Output cần trả lời
"Bạn gửi báo giá được không?"	"Chào anh/chị, tôi xin gửi báo giá đính kèm. Trân trọng."
"Khi nào có thể giao hàng?"	"Dự kiến lịch giao hàng là ngày 15/11. Trân trọng thông báo."

→ Sau fine-tune, mô hình học được **phong cách trả lời lịch sự**, dù trước đó nó đã biết tiếng Anh.

- **Quy trình Fine-Tune**



- **Source model**: mô hình đã huấn luyện tốt trên **source dataset** (tập dữ liệu lớn, tổng quát)
 - **Target model**: mô hình mới ta muốn dùng cho bài toán mới trên **target dataset** (thường nhỏ và chuyên biệt)
- Thay vì huấn luyện lại từ đầu, ta **tận dụng trọng số đã học** từ mô hình gốc.

Phần	Vai trò	Mô tả
Layer 1 đến Layer L-1 (các lớp ẩn)	Lưu trữ kiến thức chung	Những lớp này được copy nguyên từ mô hình nguồn sang mô hình đích . Đây là chỗ mà mô hình đã học các đặc trưng chung (ví dụ: cạnh, đường cong, hình dạng...).
Output layer (lớp cuối)	Dự đoán nhãn	+ Lớp này không được copy mà khởi tạo lại từ đầu , vì nhãn của bài toán mới khác so với bài toán ban đầu . + Được khởi tạo ngẫu nhiên . Vì số lớp phân loại mới thường khác (ví dụ: 1000 lớp → 2 lớp mèo/chó).
Source dataset	Dữ liệu ban đầu	Dùng để huấn luyện mô hình nguồn (pretrained model).
Target dataset	Dữ liệu mới	Dùng để tinh chỉnh mô hình cho nhiệm vụ mới.

- **Quy trình Fine-Tune**

1) Pretrain (đã làm trước): Mô hình nguồn được huấn luyện trên dữ liệu lớn (ví dụ ImageNet, Wikipedia, v.v.)

2) Copy weights: Copy các layer 1 → L-1 từ mô hình nguồn → mô hình đích. Đây là **kiến thức chung** mô hình đã học.

3) Random initialization for Output layer

4) Fine-tune

- Huấn luyện mô hình đích với **target dataset**, nhưng:
 - Các lớp 1 → L-1: **có thể khóa hoặc cập nhật nhẹ**
 - Lớp cuối: **được học mạnh nhất**